

An Inertial Newton Algorithm for Deep Learning

Camille Castera*

IRIT, Université de Toulouse, CNRS
Toulouse, France

Jérôme Bolte[†]

Toulouse School of Economics
Université de Toulouse
Toulouse, France

Cédric Févotte[†]

IRIT, Université de Toulouse, CNRS
Toulouse, France

Edouard Pauwels[†]

IRIT, Université de Toulouse, CNRS
DEEL, IRT Saint Exupéry
Toulouse, France

July 29, 2021

Abstract

We introduce a new second-order inertial optimization method for machine learning called INNA. It exploits the geometry of the loss function while only requiring stochastic approximations of the function values and the generalized gradients. This makes INNA fully implementable and adapted to large-scale optimization problems such as the training of deep neural networks. The algorithm combines both gradient-descent and Newton-like behaviors as well as inertia. We prove the convergence of INNA for most deep learning problems. To do so, we provide a well-suited framework to analyze deep learning loss functions involving tame optimization in which we study a continuous dynamical system together with its discrete stochastic approximations. We prove sublinear convergence for the continuous-time differential inclusion which underlies our algorithm. Additionally, we also show how standard optimization mini-batch methods applied to non-smooth non-convex problems can yield a certain type of spurious stationary points never discussed before. We address this issue by providing a theoretical framework around the new idea of D -criticality; we then give a simple asymptotic analysis of INNA. Our algorithm allows for using an aggressive learning rate of $o(1/\log k)$. From an empirical viewpoint, we show that INNA returns competitive results with respect to state of the art (stochastic gradient descent, ADAGRAD, ADAM) on popular deep learning benchmark problems.

Keywords. deep learning, non-convex optimization, second-order methods, dynamical systems, stochastic optimization

1 Introduction

Can we devise a learning algorithm for general deep neural networks (DNNs) featuring inertia and Newtonian directional intelligence only by means of backpropagation? In an optimization

*Corresponding author: camille.castera@protonmail.com

[†]Last three authors are listed in alphabetical order.

jargon: can we use second-order ideas in time and space for *non-smooth non-convex* optimization by only using a subgradient oracle? Before providing some answers to this question, let us have a glimpse at some fundamental optimization algorithms for training DNNs.

The backpropagation algorithm is, to this day, the fundamental block to compute gradients in deep learning (DL). It is used in most instances of the Stochastic Gradient Descent (SGD) algorithm (Robbins and Monro, 1951). The latter is powerful, flexible, capable of handling large-size problems, noise, and further comes with theoretical guarantees of many kinds. We refer to Bottou and Bousquet (2008), Moulines and Bach (2011) in a convex machine learning context and Bottou et al. (2018) for a recent account highlighting the importance of DL applications and their challenges. In the non-convex setting, recent works of Adil (2018), Davis et al. (2020) follow the *Ordinary Differential Equations (ODE) approach* introduced in Ljung (1977), and further developed in Benaïm (1999), Kushner and Yin (2003), Benaïm et al. (2005), Borkar (2009). Two research directions have been explored in order to improve SGD’s training efficiency:

- using local geometry of empirical loss functions to obtain steeper descent directions,
- using past steps history to design larger step-sizes in the present.

The first approach is akin to quasi-Newton methods while the second revolves around Polyak’s inertial method (Polyak, 1964). The latter is inspired by the following appealing mechanical thought-experiment. Consider a heavy ball evolving on the graph of the loss function (the loss function’s *landscape*), subject to gravity and stabilized by some friction effects. Friction generates energy dissipation, so that the particle will eventually reach a steady state which one hopes to be a local minimum. These two approaches are already present in the DL literature: among the most popular algorithms for training DNNs, ADAGRAD (Duchi et al., 2011) features local geometrical aspects while ADAM (Kingma and Ba, 2015) combines inertial ideas with step-sizes similar to the ones of ADAGRAD. Stochastic Newton and quasi-Newton algorithms have been considered by Martens (2010), Byrd et al. (2011, 2016) and recently reported performing efficiently on several problems (Berahas et al., 2020, Xu et al., 2020). The work of Wilson et al. (2017) demonstrates that carefully tuned SGD and heavy-ball algorithms are competitive with concurrent methods.

However, deviating from the simplicity of SGD also comes with major challenges because of the high dimensionality of DL problems and the severe absence of regularity in DL (differential regularity is generally absent, but even weaker regularity such as semi-convexity or Clarke regularity are not always available). All sorts of practical and theoretical hardships are met: computing and even defining the Hessian is delicate, inverting them is unthinkable to this day, first and second-order Taylor approximations are unavailable due to non-smoothness, and finally one has to deal with “shocks” which are inherent to inertial approaches in a non-smooth context (“corners” and “walls” in the landscape of the loss function generate velocity discontinuity). This makes in particular the study of the popular algorithms ADAGRAD and ADAM in full generality quite difficult, with recent progresses reported in Barakat and Bianchi (2021).

Our approach is inspired by the following continuous-time dynamical system introduced in Alvarez et al. (2002) and referred to as DIN (standing for “dynamical inertial Newton”):

$$\underbrace{\ddot{\theta}(t)}_{\text{Inertial term}} + \underbrace{\alpha \dot{\theta}(t)}_{\text{Friction term}} + \underbrace{\beta \nabla^2 \mathcal{J}(\theta(t)) \dot{\theta}(t)}_{\text{Newtonian effects}} + \underbrace{\nabla \mathcal{J}(\theta(t))}_{\text{Gravity effect}} = 0, \quad \text{for } t \in [0, +\infty), \quad (1)$$

where t is the time parameter which acts as a continuous epoch counter, $\mathcal{J}$ is a given loss function (e.g., the empirical loss in DL applications), for now assumed C^2 (twice-differentiable), with gradient $\nabla \mathcal{J}$ and Hessian $\nabla^2 \mathcal{J}$. It can be shown that solutions to (1) converge to critical

points of $\mathcal{J}$ (Alvarez et al., 2002). As such the discretization of (1) can support the design of algorithms that optimize $\mathcal{J}$ and that leverage inertial properties with Newton’s method. To adapt this dynamics to DL we must first overcome the computational or conceptual difficulties raised by the second-order objects $\ddot{\theta}$ and $\nabla^2 \mathcal{J}(\theta)$ appearing in (1). To do this, we propose in this paper to combine a phase-space lifting method introduced in Alvarez et al. (2002) with the use of Clarke subdifferential $\partial \mathcal{J}$. Clarke subdifferential defines a notion of differentiability for non-convex and non-smooth functions. This approach results in the study of a first-order differential inclusion in place of (1), namely,

$$\begin{cases} \dot{\theta}(t) + \beta \partial \mathcal{J}(\theta(t)) & +(\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \ni 0 \\ \dot{\psi}(t) & +(\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \ni 0 \end{cases}, \quad \text{for a.e. } t \in (0, +\infty). \quad (2)$$

This differential inclusion can then be discretized to obtain the practical algorithm INNA that is introduced in Section 2, together with a rigorous presentation of the concepts mentioned above.

Computation of the (sub)gradients and convergence proofs (in batch or mini-batch settings) typically rely on the sum-rule in smooth or convex settings, i.e., $\partial(\mathcal{J}_1 + \mathcal{J}_2) = \partial \mathcal{J}_1 + \partial \mathcal{J}_2$. Unfortunately this sum-rule does not hold in general in the non-convex setting using the standard Clarke subdifferential. Yet, many DL studies ignore the failure of the sum rule: they use it in practice, but circumvent the theoretical problem by modeling their method through simple dynamics (e.g., smooth or convex). We tackle this difficulty as is, and show that such practice can create additional spurious stationary points that are not Clarke-critical. To address this question, we introduce the notion of D -criticality. It is less stringent than Clarke-criticality and it describes more accurately real-world implementation. We then show convergence of INNA to such D -critical points. Our theoretical results are general, simple and allow for aggressive step-sizes in $o(1/\log k)$. We first provide adequate calculus rules and tame non-smooth Sard’s-like results for the new steady states we introduced. We then combine these results with a Lyapunov analysis from Alvarez et al. (2002) and the differential inclusion approximation method (Benaïm et al., 2005) to characterize the asymptotics of our algorithm similarly to Davis et al. (2020), Adil (2018). This provides a strong theoretical ground to our study since we can prove that our method converges to a connected component of the set of steady states even for networks with ReLU or other non-smooth activation functions. For the smooth deterministic dynamics, we also show that convergence in values is of the form $O(1/t)$ where t is the running time. For doing so, we provide a general result for the solutions of a family of differential inclusions having a certain type of favorable Lyapunov functions.

Our algorithm INNA shows great efficiency in practice. It has similar computational complexity to state-of-the-art methods SGD, ADAGRAD and ADAM, often achieves better training accuracy and shows good robustness to hyper-parameters selection. INNA can avoid parasitic oscillations and produce acceleration; a first illustration of the behavior of the induced dynamics is given in Figure 1 for a simple non-smooth and non-convex function in $\mathbb{R}^2$.

The rest of the paper is organized as follows. INNA is introduced in details in Section 2 and its convergence is established in Section 3. Convergence rates of the underlying continuous-time differential inclusion are obtained in Section 4. Section 5 describes experimental DL results on synthetic and real data sets (MNIST, CIFAR-10, CIFAR-100).

2 INNA: an Inertial Newton Algorithm for Deep Learning

We first introduce our functional framework, then we describe step by step the process of building INNA from (1).

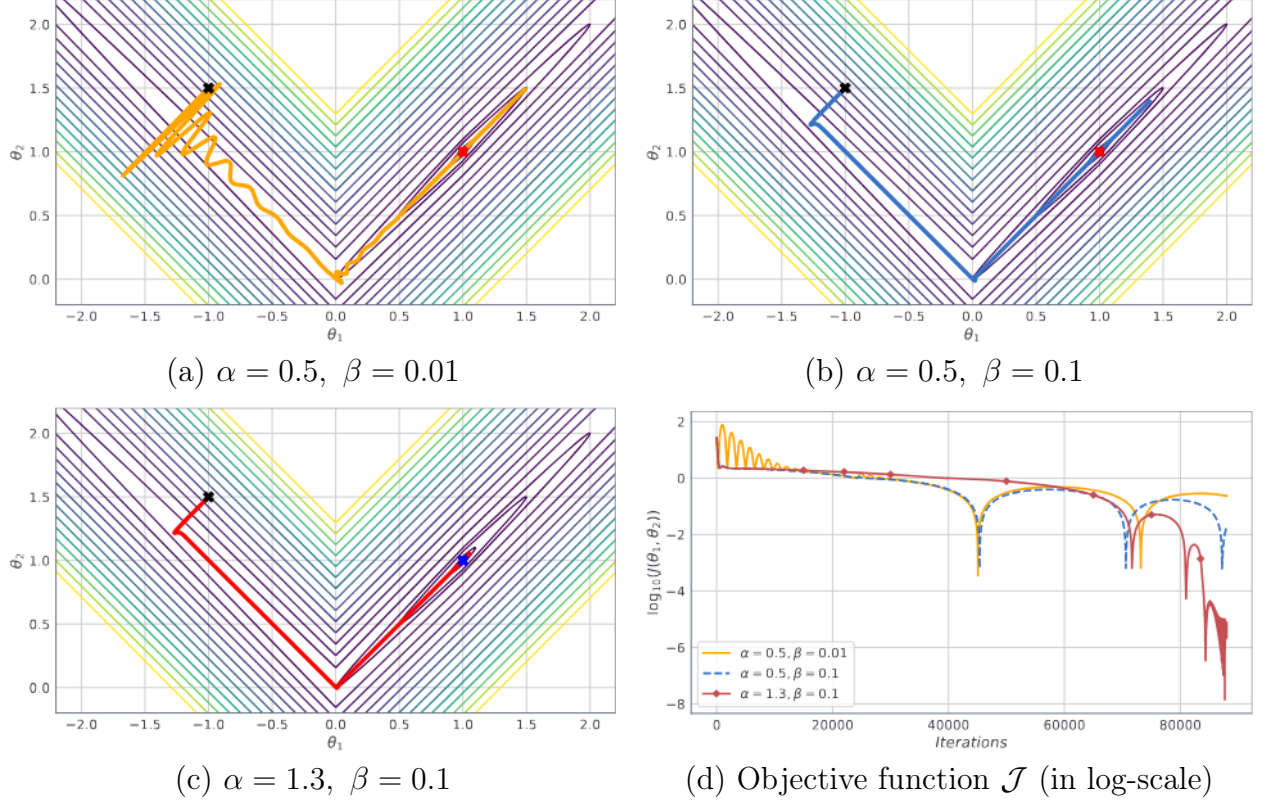


Figure 1: Illustration of INNA applied to the non-smooth function $\mathcal{J}(\theta_1, \theta_2) = 100(\theta_2 - |\theta_1|)^2 + |1 - \theta_1|$. Subplots (a-c) represent the trajectories of the parameters θ_1 and θ_2 in $\mathbb{R}^2$ for three choices of hyper-parameters α and β , see (1) for an intuitive explanation. Subplot (d) displays the values of the objective function $\mathcal{J}(\theta_1, \theta_2)$ for the three settings considered.

2.1 Neural Networks with Lipschitz Continuous Prediction Function and Loss Functions

We consider DNNs of a general type represented by a function $f : (x, \theta) \in \mathbb{R}^M \times \mathbb{R}^P \mapsto y \in \mathbb{R}^D$ that is *locally Lipschitz continuous* in θ . This includes for instance feed-forward, convolutional or residual networks used with ReLU, sigmoid, or tanh activation functions. Recall that a function $F : \mathbb{R}^P \rightarrow \mathbb{R}$ is locally Lipschitz continuous, if for any $\theta \in \mathbb{R}^P$, there exists a neighborhood V of θ and a constant $C > 0$ such that for any $\theta_1, \theta_2 \in V$,

$$|F(\theta_1) - F(\theta_2)| \leq C \|\theta_1 - \theta_2\|,$$

where $\|\cdot\|$ is any norm on $\mathbb{R}^P$. A function $F : \mathbb{R}^P \rightarrow \mathbb{R}^D$ is locally Lipschitz continuous if each of its coordinates is locally Lipschitz continuous. The variable $\theta \in \mathbb{R}^P$ is the parameter of the model (P can be very large), while $x \in \mathbb{R}^M$ and $y \in \mathbb{R}^D$ represent input and output data. For instance, the vector x may embody an image while y is a label explaining its content. Consider further a data set of N samples $(x_n, y_n)_{n=1, \dots, N}$. Training the network amounts to finding a value of the parameter θ such that, for each input data x_n of the data set, the output $f(x_n, \theta)$ of the model predicts the real value y_n with good accuracy. To do so, we follow the traditional approach of minimizing an empirical risk loss function,

$$\mathbb{R}^P \ni \theta \mapsto \mathcal{J}(\theta) = \sum_{n=1}^N l(f(x_n, \theta), y_n), \quad (3)$$

where $l : \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$ is a locally Lipschitz continuous dissimilarity measure. In the sequel, for $n \geq 1$, we will sometimes denote by $\mathcal{J}_n$ the n -th term of the sum: $\mathcal{J}_n(\theta) \triangleq l(f(x_n, \theta), y_n)$, so that $\mathcal{J} = \sum_{n=1}^N \mathcal{J}_n$. Despite the non-smoothness and the non-convexity of typical DL loss functions, they generally possess a very strong property sometimes called tameness. We now introduce this notion which is essential to obtain the convergence results of Section 3.

2.2 Neural Networks and Tameness in a Nutshell

Tameness refers to a ubiquitous geometrical property of loss functions and constraints encompassing most finite dimensional optimization problems met in practice. Prominent classes of tame objects are piecewise-linear or piecewise-polynomial objects (with finitely many pieces), and more generally, semi-algebraic objects. However, the notion is much more general, as we intend to convey below. The formal definition is given at the end of this subsection (Definition 1).

Informally, sets or functions are called tame when they can be described by a finite number of basic formulas, inequalities, or Boolean operations involving standard functions such as polynomial, exponential, or max functions. We refer to Attouch et al. (2010) for illustrations, recipes and examples within a general optimization setting or Davis et al. (2020) for illustrations in the context of neural networks. The reader is referred to Van den Dries (1998), Coste (2000), Shiota (2012) for foundational material. To apprehend the strength behind tameness it is convenient to remember that it models non-smoothness by confining the study to sets and functions which are union of smooth pieces. This is the so-called *stratification* property of tame sets and functions. It was this property which motivated the term of *tame topology*,¹ see Van den Dries (1998). In a non-convex optimization settings, the stratification property is crucial to generalize qualitative algorithmic results to non-smooth objects.

Most finite dimensional DL optimization models we are aware of yield tame loss functions $\mathcal{J}$. To understand this assertion and illustrate the wide scope of tameness assumptions, let us provide concrete examples (see also Davis et al. 2020). Assume that the DNNs under consideration are built from the following traditional components:

- the network architecture describing f is fixed with an arbitrary number of layers of arbitrary dimensions and arbitrary Directed Acyclic Graph (DAG) representing computations,
- the activation functions are among classical ones: ReLU, sigmoid, SQNL, RReLU, tanh, APL, soft plus, soft clipping, and many others including multivariate activation functions (norm, sorting), or activation functions defined piecewise with polynomials, exponential and logarithm,
- the dissimilarity function l is a standard loss such as ℓ_p norms, logistic loss or cross-entropy, or more generally a function defined piecewise using polynomials, exponentials and logarithms,

then one can easily show, by elementary quantifier elimination arguments (property (iii) below), that the corresponding loss, $\mathcal{J}$, is tame.

For the sake of completeness, we provide below the formal definition of tameness and o-minimality.

Definition 1. [o-minimal structure] (Coste, 2000, Definition 1.5) An *o-minimal* structure on $(\mathbb{R}, +, \cdot)$ is a countable collection of sets $\mathcal{O} = \{\mathcal{O}_q\}_{q \geq 1}$ where each $\mathcal{O}_q$ is itself a collection of subsets of $\mathbb{R}^q$, called *definable* subsets. They must have the following properties, for each $q \geq 1$:

¹“*La topologie modérée*” wished for by Grothendieck.

- (i) (Boolean properties) $\mathcal{O}_q$ contains the empty set, is stable by finite union, finite intersection and complementation;
- (ii) (Lifting property) if A belongs to $\mathcal{O}_q$, then $A \times \mathbb{R}$ and $\mathbb{R} \times A$ belong to $\mathcal{O}_{q+1}$.
- (iii) (Projection or quantifier elimination property) if $\Pi : \mathbb{R}^{q+1} \rightarrow \mathbb{R}^q$ is the canonical projection onto $\mathbb{R}^q$ then for any A in $\mathcal{O}_{q+1}$, the set $\Pi(A)$ belongs to $\mathcal{O}_q$.
- (iv) (Semi-algebraicity) $\mathcal{O}_q$ contains the family of algebraic subsets of $\mathbb{R}^q$, that is, every set of the form

$$\{\theta \in \mathbb{R}^q \mid \zeta(\theta) = 0\},$$

where $\zeta : \mathbb{R}^q \rightarrow \mathbb{R}$ is a polynomial function.

- (v) (Minimality property), the elements of $\mathcal{O}_1$ are exactly the finite unions of intervals and points.

A mapping $F : S \subset \mathbb{R}^m \rightarrow \mathbb{R}^q$ is said to be *definable in $\mathcal{O}$* if its graph is definable in $\mathcal{O}$ as a subset of $\mathbb{R}^m \times \mathbb{R}^q$. For illustration of o-minimality in the context of optimization one is referred to Attouch et al. (2010), Davis et al. (2020).

From now on we fix an o-minimal structure $\mathcal{O}$ and a set or a mapping definable in $\mathcal{O}$ will be called *tame*.

2.3 From DIN to INNA

We describe in this section the construction of our proposed algorithm INNA from the discretization of the second-order ODE (1).

2.3.1 Handling Non-smoothness and Non-convexity

We first show how the formalism offered by Clarke's subdifferential can be applied to generalize (1) to the non-smooth non-convex setting. Recall that the dynamical system (1) is described by,

$$\ddot{\theta}(t) + \alpha \dot{\theta}(t) + \beta \nabla^2 \mathcal{J}(\theta(t)) \dot{\theta}(t) + \nabla \mathcal{J}(\theta(t)) = 0, \quad (4)$$

where $\mathcal{J}$ is a twice-differentiable potential, $\alpha > 0$, $\beta > 0$ are two hyper-parameters and $\theta : \mathbb{R}_+ \rightarrow \mathbb{R}^P$. We cannot exploit (4) directly since in most DL applications $\mathcal{J}$ is not twice differentiable (and even not differentiable at all). We first overcome the explicit use of the Hessian matrix $\nabla^2 \mathcal{J}$ by introducing an auxiliary variable $\psi : \mathbb{R}_+ \rightarrow \mathbb{R}^P$ like in Alvarez et al. (2002). Consider the following dynamical system (defined for $\mathcal{J}$ merely differentiable),

$$\begin{cases} \dot{\theta}(t) + \beta \nabla \mathcal{J}(\theta(t)) & + (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) = 0 \\ \dot{\psi}(t) & + (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) = 0 \end{cases}, \quad \text{for a.e. } t \in (0, +\infty). \quad (5)$$

As explained in Alvarez et al. (2002), (4) is equivalent to (5) when $\mathcal{J}$ is twice differentiable. Indeed, one can rewrite (4) into (5) by introducing $\psi = -\beta\dot{\theta} - \beta^2\nabla\mathcal{J}(\theta) - (\alpha\beta - 1)\theta$. Conversely, one can substitute the first line of (5) into the second one to retrieve (4). Note however that (5) does not require the existence of second-order derivatives.

Let us now introduce a new *non-convex non-differentiable* version of (5). By Rademacher's theorem, locally Lipschitz continuous functions $\mathcal{J} : \mathbb{R}^P \rightarrow \mathbb{R}$ are differentiable almost everywhere. Denote by $\mathbf{R}$ the set of points where $\mathcal{J}$ is differentiable. Then, $\mathbb{R}^P \setminus \mathbf{R}$ has zero Lebesgue measure. It follows that for any $\theta^* \in \mathbb{R}^P \setminus \mathbf{R}$, there exists a sequence of points in $\mathbf{R}$ whose limit is this θ^* . This motivates the introduction of the subdifferential due to Clarke (1990), defined next.

Definition 2 (Clarke subdifferential of Lipschitz functions). *For any locally Lipschitz continuous function $F : \mathbb{R}^P \rightarrow \mathbb{R}$, the Clarke subdifferential of F at $\theta \in \mathbb{R}^P$, denoted $\partial F(\theta)$, is the set defined by,*

$$\partial F(\theta) = \text{conv} \left\{ v \in \mathbb{R}^P \mid \exists (\theta_k)_{k \in \mathbb{N}} \in \mathbb{R}^{\mathbb{N}}, \text{ such that } \theta_k \xrightarrow[k \rightarrow \infty]{} \theta \text{ and } \nabla F(\theta_k) \xrightarrow[k \rightarrow \infty]{} v \right\}, \quad (6)$$

where conv denotes the convex hull operator. The elements of the Clarke subdifferential are called *Clarke subgradients*.

The Clarke subdifferential is a nonempty compact convex set. It coincides with the gradient for smooth functions and with the traditional subdifferential for non-smooth convex functions. As already mentioned, and contrarily to the (sub)differential operator, it does not enjoy a sum rule.

Thanks to Definition 2, we can extend (5) to non-differentiable functions. Since $\partial \mathcal{J}(\theta)$ is a set, we no longer study a differential equation but rather a *differential inclusion*, given by,

$$\begin{cases} \dot{\theta}(t) + \beta \partial \mathcal{J}(\theta(t)) & + (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \ni 0 \\ \dot{\psi}(t) & + (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \ni 0 \end{cases}, \quad \text{for a.e. } t \in (0, +\infty). \quad (7)$$

For a given initial condition $(\theta_0, \psi_0) \in \mathbb{R}^P \times \mathbb{R}^P$, we call *solution* (or *trajectory*) of this system any absolutely continuous curve (θ, ψ) from $\mathbb{R}_+$ to $\mathbb{R}^P \times \mathbb{R}^P$ for which $(\theta(0), \psi(0)) = (\theta_0, \psi_0)$ and (7) holds. We recall that absolute continuity amounts to the fact that θ is differentiable almost everywhere with integrable derivative and,

$$\theta(t) - \theta(0) = \int_0^t \dot{\theta}(s) \, ds, \quad \text{for } t \in [0, +\infty).$$

Due to the properties of the Clarke subdifferential, existence of a solution to differential inclusions such as (7) is ensured, see Aubin and Cellina (2012); note however that uniqueness of the solution does not hold in general. We will now use the structure of (7) to build a new algorithm to train DNNs.

2.3.2 Discretization of the Differential Inclusion

To obtain the basic form of our algorithm, we discretize (7) according to the classical explicit Euler method. Given (θ, ψ) a solution of (7) and any time t_k , set $\theta_k = \theta(t_k)$ and $\psi_k = \psi(t_k)$. Then, at time $t_{k+1} = t_k + \gamma_k$ with γ_k positive small, one can approximate $\dot{\theta}(t_{k+1})$ and $\dot{\psi}(t_{k+1})$ by

$$\dot{\theta}(t_{k+1}) \simeq \frac{\theta_{k+1} - \theta_k}{\gamma_k}, \quad \dot{\psi}(t_{k+1}) \simeq \frac{\psi_{k+1} - \psi_k}{\gamma_k}.$$

This discretization yields the following algorithm,

$$\begin{cases} v_k & \in \partial \mathcal{J}(\theta_k) \\ \theta_{k+1} & = \theta_k + \gamma_k \left((\frac{1}{\beta} - \alpha)\theta_k - \frac{1}{\beta}\psi_k - \beta v_k \right) \\ \psi_{k+1} & = \psi_k + \gamma_k \left((\frac{1}{\beta} - \alpha)\theta_k - \frac{1}{\beta}\psi_k \right) \end{cases} \quad (8)$$

Although the algorithm above is well-defined for our problem, it is not suited to DL. First, the computation of $\partial \mathcal{J}(\theta_k)$ is generally not possible since there is no general operational calculus for Clarke's subdifferential; secondly, a mini-batch strategy must be designed to cope with the large dimension of DL problems which makes the absence of sum rule even more critical.

The next section is meant to address these issues and design a practical algorithm.

2.3.3 INNA Algorithm and a New Notion of Steady States

In order to compute or approximate the subdifferential of $\mathcal{J}$ at each iteration and to cope with large data sets, $\mathcal{J}$ can be approximated by mini-batches, reducing the memory footprint and computational cost of evaluation. For any $B \subset \{1, \dots, N\}$, let us define

$$\mathcal{J}_B: \theta \mapsto \sum_{n \in B} l(f(x_n, \theta), y_n). \quad (9)$$

Unlike in the differentiable case, subgradients do not in general sum up to a subgradient of the sum, that is $\partial \mathcal{J}_B(\theta) \neq \sum_{n \in B} \partial l(f(x_n, \theta), y_n)$ in general. To see this, take for example $0 = |\cdot| - |\cdot|$, the Clarke subgradient of this function at 0 is $\{0\}$, whereas $\partial(|0|) + \partial(-|0|) = [-1, 1] + [-1, 1] = [-2, 2]$. Standard DL solvers use backpropagation algorithms which implement smooth calculus on non-smooth and non-convex objects. Due to the absence of qualification conditions, *the resulting objects are not Clarke subgradients in general*. In order to match the real-world practice of DL, we introduce a notion of steady states that corresponds to the stationary points generated by a generic mini-batch approach. As we shall see, this allows both for practical applications and convergence analysis (despite the sum rule failure for Clarke subdifferential). We emphasize once more that our goal is to *capture the stationary points that are actually met in practice*.

For any $B \subset \{1, \dots, N\}$, we introduce the following objects,

$$D\mathcal{J}_B = \sum_{n \in B} \partial [l(f(x_n, \cdot), y_n)], \quad D\mathcal{J} = \sum_{n=1}^N \partial [l(f(x_n, \cdot), y_n)]. \quad (10)$$

Observe that, for each B , we have $D\mathcal{J}_B \supset \partial \mathcal{J}_B$ and that $\mathcal{J}_B$ is differentiable almost everywhere with $D\mathcal{J}_B = \partial \mathcal{J}_B = \{\nabla \mathcal{J}_B\}$, see Clarke (1990). In particular $D\mathcal{J} = \partial \mathcal{J}$ almost everywhere so that the potential differences with the Clarke subgradient occur on a negligible set. When $\mathcal{J}$ is tame the equality even hold on the complement of a finite union of manifolds of dimension strictly lower than P —use the classical stratification results for o-minimal structures, Coste (2000). A point satisfying $D\mathcal{J}(\theta) \ni 0$ will be called *D-critical*. Note that Clarke-critical points ($0 \in \partial \mathcal{J}$) are *D-critical* points but that the converse is not true. This terminology is motivated by favorable properties: sum and chain rules along curves (see Lemmas 3.2 and 3.3 below) and the existence of a tame Sard’s theorem (see Lemma 3.4). To our knowledge, this notion of a steady state has not previously been used in the literature and a direct approach modeling the mini-batch practice has never been considered before.² While this notion is needed for the theoretical analysis, one should keep in mind that $D\mathcal{J}$ is actually what is computed numerically provided that the automatic differentiation library returns a Clarke subgradient. This computation is usually done with a backpropagation algorithm, similarly to the seminal method of Rumelhart and Hinton (1986).

Ultimately, one can rewrite (7) by replacing $\partial \mathcal{J}$ by $D\mathcal{J}$, which yields a differential inclusion adapted to study mini-batch approximations of non-smooth loss functions $\mathcal{J}$. This reads,

$$\begin{cases} \dot{\theta}(t) + \beta D\mathcal{J}(\theta(t)) & +(\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \ni 0 \\ \dot{\psi}(t) & +(\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \ni 0 \end{cases}, \quad \text{for a.e. } t \in (0, +\infty). \quad (11)$$

Discretizing this system gives a workable version of INNA. Let us consider a sequence $(B_k)_{k \in \mathbb{N}}$ of nonempty subsets of $\{1, \dots, N\}$, chosen independently and uniformly at random with

²Following the first arXiv preprint of this work (Castera et al., 2019), Bolte and Pauwels (2020a) have further developed the present ideas and in particular the connection to the backpropagation algorithm.

replacement, and a sequence of positive step-sizes $(\gamma_k)_{k \in \mathbb{N}}$. For a given initialization $(\theta_0, \psi_0) \in \mathbb{R}^P \times \mathbb{R}^P$, at iteration $k \geq 1$, our algorithm reads,

$$(\text{INNA}) \quad \begin{cases} v_k & \in D\mathcal{J}_{\mathbf{B}_k}(\theta_k) \\ \theta_{k+1} & = \theta_k + \gamma_k \left(\left(\frac{1}{\beta} - \alpha \right) \theta_k - \frac{1}{\beta} \psi_k - \beta v_k \right) \\ \psi_{k+1} & = \psi_k + \gamma_k \left(\left(\frac{1}{\beta} - \alpha \right) \theta_k - \frac{1}{\beta} \psi_k \right) \end{cases} \quad (12)$$

Here again $\alpha > 0$ and $\beta > 0$ are hyper-parameters of the algorithm. The mini-batch procedure forms a stochastic approximation of the deterministic dynamics obtained by choosing $\mathbf{B}_k = \{1, \dots, N\}$, i.e., when $\mathcal{J}_{\mathbf{B}_k} = \mathcal{J}$ (batch version). This can be seen by observing that the vectors v_k above may be written as $v_k = \tilde{v}_k + \eta_k$, where $\tilde{v}_k \in D\mathcal{J}(\theta_k)$ and η_k compensates for the missing subgradients and can be seen as a zero-mean noise. Hence, INNA admits the following general abstract stochastic formulation,

$$\begin{cases} w_k & \in D\mathcal{J}(\theta_k) \\ \theta_{k+1} & = \theta_k + \gamma_k \left(\left(\frac{1}{\beta} - \alpha \right) \theta_k - \frac{1}{\beta} \psi_k - \beta w_k + \xi_k \right) \\ \psi_{k+1} & = \psi_k + \gamma_k \left(\left(\frac{1}{\beta} - \alpha \right) \theta_k - \frac{1}{\beta} \psi_k \right) \end{cases} \quad (13)$$

where $(\xi_k)_{k \in \mathbb{N}}$ is a martingale difference noise sequence adapted to the filtration induced by (random) iterates up to k . While (12) is the version implemented in practice, its equivalent form (13) is more convenient for the convergence analysis of the next section. We stress that the equivalence between (12) and (13) relies on the use of $D\mathcal{J}$ and would not hold with the use of $\partial\mathcal{J}$ as in (8).

INNA in its general and practical form is summarized in Table 1.

Inertial Newton Algorithm for Deep Learning (INNA)

Objective function: $\mathcal{J} = \sum_{n=1}^N \mathcal{J}_n$, with $\mathcal{J}_n: \mathbb{R}^P \mapsto \mathbb{R}$ locally Lipschitz.

Hyper-parameters: (α, β) positive.

Mini-batches: $(\mathbf{B}_k)_{k \in \mathbb{N}}$, nonempty subsets of $\{1, \dots, N\}$.

step-sizes: $(\gamma_k)_{k \in \mathbb{N}}$ positive.

Initialization: $(\theta_0, \psi_0) \in \mathbb{R}^P \times \mathbb{R}^P$.

For $k \in \mathbb{N}$:

$$\begin{cases} v_k & \in \sum_{n \in \mathbf{B}_k} \partial[\mathcal{J}_n(\theta_k)] \\ \theta_{k+1} & = \theta_k + \gamma_k \left(\left(\frac{1}{\beta} - \alpha \right) \theta_k - \frac{1}{\beta} \psi_k - \beta v_k \right) \\ \psi_{k+1} & = \psi_k + \gamma_k \left(\left(\frac{1}{\beta} - \alpha \right) \theta_k - \frac{1}{\beta} \psi_k \right) \end{cases}$$

Table 1: INNA in a nutshell.

3 Convergence Results for INNA

We first state our main result.

3.1 Main result: Accumulation Points of INNA are Critical

We now study the convergence of INNA. The main idea here is to prove that the discrete algorithm (12) asymptotically behaves like the solutions of the continuous differential inclusion (11). In addition to tameness, our main assumption is the following:

Assumption 1 (Stochastic approximation). *The sets $(\mathbf{B}_k)_{k \in \mathbb{N}}$ are taken independently uniformly at random with replacement. The step-size sequence γ_k is positive with $\sum_k \gamma_k = +\infty$ and satisfies $\gamma_k = o\left(\frac{1}{\log k}\right)$, that is $\limsup_{k \rightarrow +\infty} |\gamma_k \log k| = 0$.*

Typical admissible choices are $\gamma_k = C(k+1)^{-a}$ with $a \in (0, 1]$, $C > 0$. The main theoretical result of the paper follows.

Theorem 3.1 (INNA converges to the set of D -critical points of $\mathcal{J}$). *Assume that for $n \in \{1, \dots, N\}$, each $\mathcal{J}_n$ is locally Lipschitz continuous, tame and that the step-sizes satisfy Assumption 1. Set an initial condition (θ_0, ψ_0) and assume that there exists $M > 0$ such that $\sup_{k \geq 0} \|(\theta_k, \psi_k)\| \leq M$ almost surely, where $(\theta_k, \psi_k)_{k \in \mathbb{N}}$ are generated by INNA. Then, almost surely, any accumulation point $\bar{\theta}$ of the sequence $(\theta_k)_{k \in \mathbb{N}}$ satisfies $D\mathcal{J}(\bar{\theta}) \ni 0$. In addition $(\mathcal{J}(\theta_k))_{k \in \mathbb{N}}$ converges.*

Before proving Theorem 3.1, we will first make some comments and illustrate this result.

3.2 Comments on the Results of Theorem 3.1

- On the step-sizes. First, Assumption 1 offers much more flexibility than the usual $O(1/\sqrt{k})$ assumption commonly used for SGD. We leverage the boundedness assumption on the norms of (θ_k, ψ_k) , the local Lipschitz continuity and finite-sum structure of $\mathcal{J}$, so that the noise is actually uniformly bounded and hence sub-Gaussian, allowing for much larger step-sizes than in the more common bounded second moment setting. See Benaïm et al. (2005, Remark 1.5) and Benaïm (1999) for more details. The interest of this aggressive strategy is highlighted in Figure 4 of the experimental section.
- On the scope of the theorem. Our result actually holds for general locally Lipschitz continuous tame functions with finite-sum structure and for the general stochastic process under uniformly bounded martingale increment noise. We do not use any other specific structure of DL loss functions. Other variants could be considered depending on the assumptions on the noise, see Benaïm et al. (2005).
- On D -criticality. The result of Theorem 3.1 states that the bounded discrete trajectories of INNA are attracted by the D -critical points. Recall that D -critical points include local minimizers and thus our theoretical finding agrees with our empirical observations that most initializations lead to “valuable weights” θ and to efficient training. In particular for smooth networks where $\mathcal{J}$ is differentiable, limit points of INNA are simply critical points of $\mathcal{J}$. The reader should however remember that when the algorithm is initialized on the D -critical set, the algorithm is stationary as well, *even when the initialization is non-Clarke critical*. This last point shows that D -points are not introduced to simplify the analysis but to *sharply model the use of mini-batch methods on non-convex and non-smooth problems*. Hopefully, in practice one can expect to avoid such points with overwhelming probability. Indeed, Bolte and Pauwels (2020b) proved that SGD converges with probability one to the set of Clarke-critical points. In other words, D -critical points that are not Clarke-critical

are not seen by the dynamics (see also Bianchi et al. 2020). The key argument is to prove that the set of initializations and step-sizes that leads the algorithm to reach points where $D\mathcal{J} \neq \partial\mathcal{J}$ has zero-measure. The same result can be hoped for INNA.

- On the boundedness assumption. The bounded assumption on the iterates is a classical assumption for first or second-order algorithms, see for instance Davis et al. (2020), Duchi and Ruan (2018). When using deterministic algorithms (i.e., without mini-batch approximations), properties such as the coercivity of $\mathcal{J}$ can be sufficient to remove the boundedness assumption for descent algorithms. This does not remain true when dealing with mini-batch approximations, yet, in the case of INNA, the coercivity of $\mathcal{J}$ would guarantee at least that the solutions of the continuous underlying differential inclusion (11) remain bounded. Indeed, we will prove in Section 3.4 that for any solution (θ, ψ) of (11), the function $E(\theta(t), \psi(t)) \triangleq 2(1 + \alpha\beta)\mathcal{J}(\theta(t)) + \left\|(\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t)\right\|^2$ is decreasing in time (see Lemma 3.6 hereafter). As a consequence, we cannot have $\mathcal{J}(\theta(t)) \xrightarrow[t \rightarrow \infty]{} \infty$ so $\|\theta(t)\| \not\xrightarrow[t \rightarrow \infty]{} \infty$ due to the coercivity of $\mathcal{J}$. In addition this guarantees $\|\psi(t)\| \not\xrightarrow[t \rightarrow \infty]{} \infty$ as well. However, DL loss functions are not coercive in general and studying the boundedness issue in DL or even for non-convex semi-algebraic problems is far beyond the scope of this paper. Let us however mention that it is not uncommon to project the iterates on a given large ball to ensure boundedness; this is a matter for future research.

3.3 Preliminary Variational Results

Prior to proving Theorem 3, we extend some results known for the Clarke subdifferential of tame functions to the operator D that we previously introduced. First, we recall a useful result of Davis et al. (2020) which follows from the projection formula in Bolte et al. (2007b).

Lemma 3.2 (Chain rule for the Clarke subdifferential). *Let $\mathcal{J} : \mathbb{R}^P \rightarrow \mathbb{R}$ be a locally Lipschitz continuous tame function, then $\mathcal{J}$ admits a chain rule, meaning that for all absolutely continuous curves $\theta : \mathbb{R}_+ \rightarrow \mathbb{R}^P$, $\mathcal{J} \circ \theta$ is differentiable a.e. and for a.e. $t \geq 0$,*

$$\frac{d\mathcal{J}}{dt}(\theta(t)) = \langle \dot{\theta}(t), \partial\mathcal{J}(\theta(t)) \rangle = \langle \dot{\theta}(t), v \rangle, \quad \forall v \in \partial\mathcal{J}(\theta(t)). \quad (14)$$

Note that, even though $\mathcal{J}$ is non-differentiable on $\mathbb{R}^P$, the function $t \mapsto \mathcal{J}(\theta(t))$ is differentiable for a.e. $t > 0$. Indeed, as introduced in Section 2.3.1, an absolutely continuous curve from $t \geq 0$ to $\mathbb{R}^P$ is differentiable for a.e. $t > 0$. This, combined with the chain-rule of Lemma 3.2 allows differentiating $\mathcal{J} \circ \theta$ for a.e. $t > 0$ whenever $\mathcal{J}$ is tame and locally Lipschitz continuous. Additionally, notice that the value of $\frac{d\mathcal{J}}{dt}(\theta(t))$ in (14) does not depend on the element v taken in $\partial\mathcal{J}(\theta(t))$ which justifies the notation $\langle \dot{\theta}(t), \partial\mathcal{J}(\theta(t)) \rangle$.

Consider now a function with an additive finite-sum structure (such as in DL):

$$\mathcal{J} : \mathbb{R}^P \ni \theta \mapsto \sum_{n=1}^N \mathcal{J}_n(\theta), \quad (15)$$

where each $\mathcal{J}_n : \mathbb{R}^P \mapsto \mathbb{R}$ is locally Lipschitz continuous and tame. We define for any $\theta \in \mathbb{R}^P$

$$D\mathcal{J}(\theta) = \sum_{n=1}^N \partial\mathcal{J}_n(\theta).$$

The following lemma is a direct generalization of the above chain rule.

Lemma 3.3 (Chain rule for $D\mathcal{J}$). *Let $\mathcal{J}$ be a sum of tame functions like in (15). Let $c: [0, 1] \mapsto \mathbb{R}^P$ be an absolutely continuous curve so that $t \mapsto \mathcal{J}(c(t))$ is differentiable almost everywhere. For a.e. $t \in [0, 1]$, and for all $v \in D\mathcal{J}(c(t))$,*

$$\frac{d}{dt}\mathcal{J}(c(t)) = \langle v, \dot{c}(t) \rangle.$$

Proof. By local Lipschitz continuity and absolute continuity, each $\mathcal{J}_n$ is differentiable almost everywhere and Lemma 3.2 can be applied:

$$\frac{d}{dt}\mathcal{J}_n(c(t)) = \langle v_n, \dot{c}(t) \rangle, \text{ for all } v_n \in \partial\mathcal{J}_n(c(t)) \text{ and for a.e. } t \geq 0.$$

Thus

$$\frac{d}{dt}\mathcal{J}(c(t)) = \sum_{n=1}^N \frac{d}{dt}\mathcal{J}_n(c(t)) = \sum_{n=1}^N \langle v_n, \dot{c}(t) \rangle,$$

for any $v_n \in \partial\mathcal{J}_n(c(t))$, for all $n = \{1, \dots, N\}$, and for a.e. $t \geq 0$. This proves the desired result. $\square$

We finish this section with a Sard lemma for D -critical values, in the spirit of Bolte et al. (2007b).

Lemma 3.4 (A Sard's theorem for tame D -critical values). *Let,*

$$\mathbf{S} = D\text{-crit} \triangleq \{\theta \in \mathbb{R}^P \mid D\mathcal{J}(\theta) \ni 0\},$$

then $\mathcal{J}(\mathbf{S})$ is finite.

Proof. The set $\mathbf{S}$ is tame and hence it has a finite number of connected components. It is sufficient to prove that $\mathcal{J}$ is constant on each connected component of $\mathbf{S}$. Without loss of generality, assume that $\mathbf{S}$ is connected and consider $\theta_0, \theta_1 \in \mathbf{S}$. By Whitney regularity (Van den Dries, 1998, 4.15), there exists a tame continuous path Γ joining θ_0 to θ_1 . Because of the tame nature of the result, we should here conclude with only tame arguments and use the projection formula in Bolte et al. (2007b), but for convenience of readers who are not familiar with this result we use Lemma 3.2. Since Γ is tame, the monotonicity lemma (see for example Kurdyka 1998, Lemma 2) gives the existence of a finite collection of real numbers $0 = a_0 < a_1 < \dots < a_q = 1$, such that Γ is C^1 on each segment (a_{j-1}, a_j) , $j = 1, \dots, q$. Applying Lemma 3.2 to each $\Gamma|_{(a_i, a_{i+1})}$, we see that $\mathcal{J}$ is constant except perhaps on a finite number of points, thus $\mathcal{J}$ is constant by continuity. $\square$

3.4 Proof of Convergence for INNA

Our approach follows the stochastic method for differential inclusions developed in Benaïm et al. (2005) for which the differential system (11) and its Lyapunov properties play fundamental roles. The steady states of (11) are given by,

$$\mathbf{S} = \{(\theta, \psi) \in \mathbb{R}^P \times \mathbb{R}^P \mid 0 \in D\mathcal{J}(\theta), \psi = (1 - \alpha\beta)\theta\}. \quad (16)$$

These points are initialization values for which the system does not evolve and remains constant. Observe that the first coordinates of these points are D -critical for $\mathcal{J}$ and that conversely any D -critical point of $\mathcal{J}$ corresponds to a unique rest point in $\mathbf{S}$.

Definition 3 (Lyapunov function). *Let $\mathbf{A}$ be a subset of $\mathbb{R}^P \times \mathbb{R}^P$, we say that $E : \mathbb{R}^P \times \mathbb{R}^P \rightarrow \mathbb{R}$ is a Lyapunov function for the set $\mathbf{A}$ and the dynamics (11) if,*

- (i) *For any solution (θ, ψ) of (11) with initial condition $(\theta_0, \psi_0) \in \mathbb{R}^P \times \mathbb{R}^P$, we have:
 $E(\theta(t), \psi(t)) \leq E(\theta_0, \psi_0)$ a.e. on $\mathbb{R}$.*
- (ii) *For any solution (θ, ψ) of (11) with initial condition $(\theta_0, \psi_0) \in \mathbb{R}^P \times \mathbb{R}^P \setminus \mathbf{A}$, we have:
 $E(\theta(t), \psi(t)) < E(\theta_0, \psi_0)$ a.e. on $\mathbb{R}$.*

In practice, to establish that a functional is Lyapunov, one can simply use differentiation through chain rule results, with in particular Lemma 3.2. In the context of INNA, we will use Lemma 3.3. To build a Lyapunov function for the dynamics (11) and the set $\mathbf{S}$, consider the two following energy-like functions,

$$\begin{cases} E_{\min}(\theta(t), \psi(t)) &= (1 - \sqrt{\alpha\beta})^2 \mathcal{J}(\theta(t)) + \frac{1}{2} \left\| (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \right\|^2 \\ E_{\max}(\theta(t), \psi(t)) &= (1 + \sqrt{\alpha\beta})^2 \mathcal{J}(\theta(t)) + \frac{1}{2} \left\| (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \right\|^2. \end{cases} \quad (17)$$

Then the following lemma applies.

Lemma 3.5 (Differentiation along DIN trajectories). *Let (θ, ψ) be a solution of (11) with initial condition (θ_0, ψ_0) . For a.e. $t > 0$, θ and ψ are differentiable at t , (11) holds, $\frac{\dot{\theta}(t) - \dot{\psi}(t)}{\beta} \in D\mathcal{J}(\theta(t))$ and*

$$\begin{aligned} \frac{dE_{\min}}{dt}(\theta(t), \psi(t)) &= - \left\| \sqrt{\alpha}\dot{\theta}(t) - \frac{1}{\sqrt{\beta}}(\dot{\psi}(t) - \dot{\theta}(t)) \right\|^2 \\ \frac{dE_{\max}}{dt}(\theta(t), \psi(t)) &= - \left\| \sqrt{\alpha}\dot{\theta}(t) + \frac{1}{\sqrt{\beta}}(\dot{\psi}(t) - \dot{\theta}(t)) \right\|^2 \end{aligned}$$

Proof. Define $E_\lambda(\theta, \psi) = \lambda \mathcal{J}(\theta) + \frac{1}{2} \left\| (\alpha - \frac{1}{\beta})\theta + \frac{1}{\beta}\psi \right\|^2$. We aim to choose λ so that E_λ is a Lyapunov function. Because $\mathcal{J}$ is tame and locally Lipschitz continuous, using Lemma 3.3 we know that for any absolutely continuous trajectory $\theta : \mathbb{R}_+ \rightarrow \mathbb{R}^P$ and for a.e. $t > 0$,

$$\frac{d\mathcal{J}}{dt}(\theta(t)) = \langle \dot{\theta}(t), D\mathcal{J}(\theta(t)) \rangle = \langle \dot{\theta}(t), v(t) \rangle, \quad \forall v(t) \in D\mathcal{J}(\theta(t)). \quad (18)$$

Let θ and ψ be solutions of (DIN). For a.e. $t \geq 0$, we can differentiate $E_\lambda(\theta, \psi)$ to obtain

$$\begin{aligned} \frac{dE_\lambda}{dt}(\theta(t), \psi(t)) &= \lambda \langle \dot{\theta}(t), v(t) \rangle + (\alpha - \frac{1}{\beta}) \langle \dot{\theta}(t), (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \rangle \\ &\quad + \frac{1}{\beta} \langle \dot{\psi}(t), (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t) \rangle \end{aligned} \quad (19)$$

for all $v(t) \in D\mathcal{J}(\theta(t))$. Using (11), we get $\frac{1}{\beta}(\dot{\theta}(t) - \dot{\psi}(t)) \in D\mathcal{J}(\theta(t))$ and $-\dot{\psi}(t) = (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t)$ a.e. Choosing $v(t) = \frac{1}{\beta}(\dot{\theta}(t) - \dot{\psi}(t))$ yields:

$$\frac{dE_\lambda}{dt}(\theta(t), \psi(t)) = \lambda \left\langle \dot{\theta}(t), \frac{\dot{\theta}(t) - \dot{\psi}(t)}{\beta} \right\rangle - (\alpha - \frac{1}{\beta}) \langle \dot{\theta}(t), \dot{\psi}(t) \rangle - \frac{1}{\beta} \langle \dot{\psi}(t), \dot{\psi}(t) \rangle.$$

Then, expressing everything as a function of $\dot{\theta}$ and $\frac{1}{\beta}(\psi - \theta)$, one can show that a.e. on $\mathbb{R}_+$:

$$\begin{aligned} \frac{dE_\lambda}{dt}(\theta, \psi)(t) &= -\alpha \|\dot{\theta}(t)\|^2 - \beta \left\| \frac{\dot{\theta}(t) - \dot{\psi}(t)}{\beta} \right\|^2 + (\lambda - \alpha\beta - 1) \left\langle \dot{\theta}(t), \frac{\dot{\theta}(t) - \dot{\psi}(t)}{\beta} \right\rangle \\ &= - \left\| \sqrt{\alpha} \dot{\theta}(t) + \frac{\alpha\beta + 1 - \lambda}{2\sqrt{\alpha}} \frac{\dot{\theta}(t) - \dot{\psi}(t)}{\beta} \right\|^2 - \left(\beta - \frac{(\alpha\beta + 1 - \lambda)^2}{4\alpha} \right) \left\| \frac{\dot{\theta}(t) - \dot{\psi}(t)}{\beta} \right\|^2. \end{aligned}$$

We aim to choose λ so that E_λ is decreasing that is $\left(\beta - \frac{(\alpha\beta + 1 - \lambda)^2}{4\alpha} \right) > 0$. This holds whenever $\lambda \in [(1 - \sqrt{\alpha\beta})^2, (1 + \sqrt{\alpha\beta})^2]$. We choose $\lambda_{\min} = (1 - \sqrt{\alpha\beta})^2$, and $\lambda_{\max} = (1 + \sqrt{\alpha\beta})^2$, for these two values we obtain for a.e. $t > 0$,

$$\begin{cases} \dot{E}_{\lambda_{\min}}(\theta(t), \psi(t)) &= - \left\| \sqrt{\alpha} \dot{\theta}(t) + \frac{1}{\sqrt{\beta}} \left(\dot{\theta}(t) - \dot{\psi}(t) \right) \right\|^2 \\ \dot{E}_{\lambda_{\max}}(\theta(t), \psi(t)) &= - \left\| \sqrt{\alpha} \dot{\theta}(t) - \frac{1}{\sqrt{\beta}} \left(\dot{\theta}(t) - \dot{\psi}(t) \right) \right\|^2 \end{cases} \quad (20)$$

Remark finally that by definition $E_{\min} = E_{\lambda_{\min}}$ and $E_{\max} = E_{\lambda_{\max}}$. $\square$

Recall that $\mathbf{S} = \{(\theta, \psi) \in \mathbb{R}^P \times \mathbb{R}^P \mid 0 \in DJ(\theta), \psi = (1 - \alpha\beta)\theta\}$ and define $E = E_{\min} + E_{\max}$. By a direct integration argument, we obtain the following lemma.

Lemma 3.6 (E is Lyapunov function for INNA with respect to $\mathbf{S}$). *For all $(\theta_0, \psi_0) \notin \mathbf{S}$ and for any solution (θ, ψ) with initial condition (θ_0, ψ_0) ,*

$$E(\theta(t), \psi(t)) < E(\theta_0, \psi_0), \text{ for a.e. } t > 0. \quad (21)$$

We are now in position to provide the desired proof.

Proof of Theorem 3.1 Lemmas 3.5 and 3.6 state that E is a Lyapunov function for the set $\mathbf{S}$ and the dynamics (11). Let $\mathbf{C} = \{\theta \in \mathbb{R}^P \mid (\theta, \psi) \in \mathbf{S}\}$ which is actually the set of D -critical points of $\mathcal{J}$. Using Lemma 3.4 of Section 3.3, $\mathcal{J}(\mathbf{C})$ is finite. Moreover, since $E(\theta, \psi) = 2(1 + \alpha\beta)\mathcal{J}(\theta)$ for all $(\theta, \psi) \in \mathbf{S}$, E takes a finite number of values on $\mathbf{S}$, and in particular, $E(\mathbf{S})$ has empty interior.

Denote by $\mathbf{L}$ the set of accumulation points of the sequences $((\theta_k, \psi_k))_{k \in \mathbb{N}}$ produced by (12) starting at (θ_0, ψ_0) and $\mathbf{L}_1$ its projection on $\mathbb{R}^P \times \{0\}$. We have the 3 following properties:

- By assumption, we have $\|(\theta_k, \psi_k)\| \leq M$ almost surely, for all $k \in \mathbb{N}$.
- By local Lipschitz continuity $\partial \mathcal{J}_{\mathbf{B}}(\theta)$ is uniformly bounded for $\|\theta\| \leq M$ and any $\mathbf{B} \subset \{1, \dots, N\}$, hence the centered noise $(\xi_k)_{k \in \mathbb{N}}$ is a uniformly bounded martingale difference sequence.
- By Assumption 1, the sequence $(\gamma_k)_{k \in \mathbb{N}}$ is chosen such that $\gamma_k = o(\frac{1}{\log k})$ (see Section 3.2).

Then the sufficient conditions of Remark 1.5 of Benaïm et al. (2005) state that the discrete process $(\theta_k, \psi_k)_{k \in \mathbb{N}}$ asymptotically behaves like the solutions of (11). We can then combine Proposition 3.27 and Theorem 3.6 of Benaïm et al. (2005), to obtain that the limit set $\mathbf{L}$ of the discrete process is contained in the set $\mathbf{S}$ where the Lyapunov has vanishing derivatives. Thus, the set $\mathbf{L}_1$ (the set of the first coordinates of all accumulation points) contains only D -critical points of $\mathcal{J}$. In addition, $E(\mathbf{L})$ is a singleton, and for all $(\theta, \psi) \in \mathbf{S}$, we have $E(\theta, \psi) = \mathcal{J}(\theta)$, so $\mathcal{J}(\mathbf{L}_1)$ is also a singleton and the theorem follows.

4 Towards Convergence Rates for INNA

In the previous section, connecting INNA to (11) was one of the keys to prove the convergence of the discrete dynamics. Let us now focus on the continuous dynamical system (7) in the deterministic case where $\mathcal{J}$ and $\partial\mathcal{J}$ are not approximated anymore—we thus no longer use $D\mathcal{J}$ although this would be possible but would require more technical proofs. In this section and in this section only, we pertain to loss functions $\mathcal{J}$ that are real semi-algebraic (a particular case of tame functions).³ Recall that a set is called semi-algebraic if it is a finite union of sets of the form,

$$\{\theta \in \mathbb{R}^P \mid \zeta(\theta) = 0, \zeta_i(\theta) < 0\}$$

where ζ, ζ_i are real polynomial functions. A function is called semi-algebraic if its graph is semi-algebraic.

We will characterize the convergence rate of the solutions of the continuous-time system (7) to critical points. Let us first introduce an essential mechanism to obtain such convergence rates: the Kurdyka-Łojasiewicz (KL) property.

4.1 The Non-smooth Kurdyka-Łojasiewicz Property for the Clarke Subdifferential

The non-smooth Kurdyka-Łojasiewicz (KL) property, as introduced in (Bolte et al., 2010), is a measure of “amenability to sharpness” (as illustrated at the end of Section 4.3). Here we provide a uniform version for the Clarke subdifferential of semi-algebraic functions as in Bolte et al. (2007b) and Bolte et al. (2014). In the sequel we denote by “dist” any given distance on $\mathbb{R}^P$.

Lemma 4.1 (Uniform Non-smooth KL Property for the Clarke Subdifferential). *Let $\mathbf{K}$ be a nonempty compact set and let $L : \mathbb{R}^P \rightarrow \mathbb{R}$ be a semi-algebraic locally Lipschitz continuous function. Assume that L is constant on $\mathbf{K}$, with value L^* . Then there exist $\varepsilon > 0$, $\delta > 0$, $a \in (0, 1)$ and $\rho > 0$ such that, for all*

$$v \in \{v \in \mathbb{R}^P \mid \text{dist}(v, \mathbf{K}) < \varepsilon\} \cap \{v \in \mathbb{R}^P \mid L^* < L(v) < L^* + \delta\},$$

it holds that,

$$\rho(1 - a) (L(v) - L(\bar{v}))^{-a} \text{dist}(0, \partial L(v)) > 1. \quad (22)$$

The proof directly follows from the general inequality provided in Bolte et al. (2007b) or the local result of Bolte et al. (2007b) with the compactness arguments of Bolte et al. (2014, Lemma 6). In the sequel, we make an abuse of notation by writing $\|\partial\mathcal{J}(\cdot)\| \triangleq \text{dist}(0, \partial\mathcal{J}(\cdot))$. To obtain a convergence rate we will use inequality (22) on the Lyapunov function E . But first we state a general result of convergence that is built around the KL property.

4.2 A General Asymptotic Rate Result

We state a general theorem that leads to the existence of a convergence rate. This theorem will hold in particular for (7). We start by stating the result.

³We could extend the results of this section to more general objects including analytic functions on bounded sets. The semi-algebraicity assumption is made here for the sake of clarity.

Theorem 4.2. *Let $X : [0, +\infty) \rightarrow \mathbb{R}^P$ be a bounded absolutely continuous trajectory and let $L : \mathbb{R}^P \rightarrow \mathbb{R}$ be a semi-algebraic locally Lipschitz continuous function. If there exists $c_1 > 0$ such that for a.e. $t > 0$,*

$$\frac{dL}{dt}(X(t)) \leq -c_1 \|(\partial L)(X(t))\|^2, \quad (\text{i})$$

then $L(X(t))$ converges to a limit value L^ and,*

$$|L(X(t)) - L^*| = O\left(\frac{1}{t}\right).$$

If in addition there exists $c_2 > 0$ such that for a.e. $t > 0$,

$$c_2 \|\dot{X}(t)\| \leq \|(\partial L)(X(t))\|, \quad (\text{ii})$$

then, X converges to a critical point of L with a rate of the form $O(1/t^b)$ with $b > 0$.⁴

Proof. We first prove the convergence of $L(X(\cdot))$. Suppose that (i) holds. Since X is bounded and L is continuous, $L(X(\cdot))$ is bounded. Moreover, from (i), $L(X(\cdot))$ is decreasing, so it converges to some value L^* . To simplify suppose $L \geq 0$ and $L^* = 0$. Define,

$$\mathbf{l} = \{x \in \mathbb{R}^P \mid L(x) = 0\}.$$

Suppose first that there exists $s \geq 0$, such that $X(s) \in \mathbf{l}$. Since $L(X(\cdot))$ is decreasing with limit 0, then for all $t \geq s$, $L(X(t)) = 0$ and the convergence rate holds true.

Let us thus assume that for all $t \geq 0$, $L(X(t)) > 0$. The trajectory X is bounded in $\mathbb{R}^P$, hence there exists a compact set $\mathbf{C} \subset \mathbb{R}^P$ such that $X(t) \in \mathbf{C}$ for all $t \geq 0$. Define $\mathbf{K} = \mathbf{l} \cap \mathbf{C}$. It is a compact set since $\mathbf{l}$ is closed (by continuity of L) and $\mathbf{C}$ is compact. Moreover, L is constant on $\mathbf{K}$. As such by Lemma 4.1, there exist $\varepsilon > 0$, $\delta > 0$, $a \in (0, 1)$ and a constant $\rho > 0$ such that for all

$$v \in \{v \in \mathbb{R}^P, \text{dist}(v, \mathbf{K}) < \varepsilon\} \cap \{0 < L(v) < \delta\},$$

it holds that

$$\rho(1-a)(L(v))^{-a} \text{dist}(0, \partial L(v)) > 1.$$

We have $L(X(t)) \rightarrow 0$ so there exists $t_0 \geq 0$ such that for all $t \geq t_0$, $0 < L(X(t)) < \delta$. Without loss of generality, we assume $t_0 = 0$. Similarly, we have $\text{dist}(X(t), \mathbf{K}) \rightarrow 0$, so we may assume that for all $t \geq 0$, $\text{dist}(X(t), \mathbf{K}) < \varepsilon$. Thus, for all $t \geq 0$,

$$\rho(1-a)L(X(t))^{-a} \|\partial L(X(t))\| > 1.$$

Going back to assumption (i), for a.e. $t > 0$, one has

$$\frac{dL}{dt}(X(t)) \leq -c_1 \|(\partial L)(X(t))\|^2,$$

but the KL property implies that for a.e. $t > 0$,

$$-\|\partial L(X(t))\|^2 < -\frac{1}{\rho^2(1-a)^2} L(X(t))^{2a}.$$

Therefore,

$$\frac{dL}{dt}(X(t)) < -\frac{c_1}{\rho^2(1-a)^2} L(X(t))^{2a}.$$

⁴In some cases we even have linear rates or finite convergence as detailed in the proof.

We consider two cases depending on the value of a . If $0 < a \leq 1/2$, then for t large, $L(X(t)) < 1$ so $-L(X(t))^{2a} < -L(X(t))$ and hence,

$$\frac{dL}{dt}(X(t)) < -\frac{c_1}{\rho^2(1-a)^2}L(X(t)),$$

so we obtain a linear rate. When $1/2 < a < 1$, we have for a.e. $t > 0$,

$$L(X(t))^{-2a} \frac{d}{dt} L(X(t)) = \frac{1}{1-2a} \frac{d}{dt} L(X(t))^{1-2a} < -\frac{c_1}{(\rho^2(1-a)^2)}, \quad (23)$$

with $1-2a < 0$. We can integrate from 0 to $t > 0$:

$$L(X(t))^{1-2a} > \frac{(2a-1)c_1}{\rho^2(1-a)^2}t + L(X(0))^{1-2a} > \frac{(2a-1)c_1}{\rho^2(1-a)^2}t.$$

Since $\frac{1}{1-2a} < -1$, one obtains a convergence rate of the form $O\left(t^{\frac{1}{1-2a}}\right)$. In both cases the rate is at least $O\left(\frac{1}{t}\right)$.

We assume now that both (i) and (ii) holds and prove convergence of the trajectory with a convergence rate. Let $t > s > 0$, by the fundamental theorem of calculus and the triangular inequality,

$$\|X(t) - X(s)\| \leq \left\| \int_s^t \dot{X}(\tau) d\tau \right\| \leq \int_s^t \|\dot{X}(\tau)\| d\tau. \quad (24)$$

We wish to bound $\|\dot{X}\|$ using L . Using the chain rule (Lemma 3.2 of Section 3.3), for a.e. $\tau > 0$,

$$\frac{d}{d\tau} L(X(\tau))^{1-a} = (1-a)L(X(\tau))^{-a} \langle \dot{X}(\tau), (\partial L)(X(\tau)) \rangle. \quad (25)$$

Then, from (i), we deduce that for a.e. $\tau > 0$,

$$\langle \dot{X}(\tau), (\partial L)(X(\tau)) \rangle = \frac{dL}{d\tau}(X(\tau)) \leq -c_1 \|(\partial L)(X(\tau))\|^2, \quad (26)$$

so

$$\frac{d}{d\tau} L(X(\tau))^{1-a} \leq -c_1(1-a)L(X(\tau))^{-a} \|(\partial L)(X(\tau))\|^2. \quad (27)$$

The KL property (22) implies that for a.e. $\tau > 0$,

$$-(1-a)L(X(\tau))^{-a} \|(\partial L)(X(\tau))\| < -\frac{1}{\rho}. \quad (28)$$

Putting this in (27) and using assumption (ii) we finally obtain

$$\frac{d}{d\tau} L(X(\tau))^{1-a} < -\frac{c_1}{\rho} \|(\partial L)(X(\tau))\| \leq -\frac{c_1 c_2}{\rho} \|\dot{X}(\tau)\|. \quad (29)$$

We can use that in (24),

$$\begin{aligned} \|X(t) - X(s)\| &\leq -\frac{\rho}{c_1 c_2} \int_s^t \frac{d}{d\tau} L(X(\tau))^{1-a} d\tau \\ &= \frac{\rho}{c_1 c_2} (L(X(s))^{1-a} - L(X(t))^{1-a}). \end{aligned} \quad (30)$$

Then, using the convergence rate we already proved for L , we deduce that the Cauchy criterion holds for X inside the compact (hence complete) subset $\mathbf{C} \subset \mathbb{R}^P$ containing the trajectory. Thus, X converges, and from (i) we have that $\liminf_{t \rightarrow +\infty} \|\partial L(X(t))\| = 0$ because ∂L has closed graph. This shows that the limit is a critical point of L . Finally, taking the limit in (30) and using the convergence rate of L we obtain a rate for X as well. $\square$

Remark 1. Theorem 4.2 takes the form of a general recipe to obtain a convergence rate since it may be applied in many cases, to curves or flows, provided that a convenient Lyapunov function is given. Note also that it is sufficient for assumptions (i) and (ii) to hold only after some time $t_0 > 0$ as in such case, one could simply do a time shift to use the theorem.

4.3 Application to INNA

We now apply Theorem 4.2 to the deterministic continuous dynamical model of INNA (7).

Theorem 4.3 (Convergence rates). *Suppose that $\mathcal{J}$ is semi-algebraic locally Lipschitz continuous and lower bounded. Then, any bounded trajectory (θ, ψ) that solves (7) converges to a point $(\bar{\theta}, \bar{\psi}) \in \mathcal{S}$, with a convergence rate of the form $O(t^{-b})$ with $b > 0$. Moreover, $\mathcal{J}(\theta(t))$ converges to its limit $\bar{\mathcal{J}}$ with rate $|\mathcal{J}(\theta(t)) - \bar{\mathcal{J}}| = O(\frac{1}{t})$.*

Proof. Let (θ, ψ) be a bounded solution of (7). We would like to use Theorem 4.2 with $X = (\theta, \psi)$, and a well-chosen function. In the proof of Theorem 3.1 we proved a descent property along the trajectory for the function $E(\theta, \psi) = 2(1 + \alpha\beta)\mathcal{J}(\theta) + \left\|(\alpha - \frac{1}{\beta})\theta + \frac{1}{\beta}\psi\right\|^2$. This function is semi-algebraic, locally Lipschitz continuous, so it remains to prove that (i) and (ii) hold for E along (θ, ψ) .

For $t \geq 0$, denote $w(t) = (\alpha - \frac{1}{\beta})\theta(t) + \frac{1}{\beta}\psi(t)$, then according to Lemma 3.5 for a.e. $t > 0$,

$$\begin{aligned} \frac{dE}{dt}(\theta(t), \psi(t)) &= -\|\sqrt{\alpha}\dot{\theta}(t) - \frac{1}{\sqrt{\beta}}(\dot{\psi}(t) - \dot{\theta}(t))\|^2 - \|\sqrt{\alpha}\dot{\theta}(t) + \frac{1}{\sqrt{\beta}}(\dot{\psi}(t) - \dot{\theta}(t))\|^2 \\ &= -2\alpha\|\dot{\theta}(t)\|^2 - \frac{2}{\beta}\|\dot{\psi}(t) - \dot{\theta}(t)\|^2 = -2\alpha\|\dot{\theta}(t)\|^2 - \frac{2}{\beta}\|\beta\partial\mathcal{J}(\theta(t))\|^2 \\ &= -2\alpha\| -\beta\partial\mathcal{J}(\theta(t)) - w(t)\|^2 - 2\beta\|\partial\mathcal{J}(\theta(t))\|^2. \end{aligned} \quad (31)$$

On the other hand, by standard results on the sum rule, we have for all $(\theta, \psi) \in \mathbb{R}^P \times \mathbb{R}^P$,

$$\partial E(\theta, \psi) = 2 \begin{pmatrix} (1 + \alpha\beta)\partial\mathcal{J}(\theta) + (\alpha - \frac{1}{\beta}) \left((\alpha - \frac{1}{\beta})\theta + \frac{1}{\beta}\psi \right) \\ \frac{1}{\beta} \left((\alpha - \frac{1}{\beta})\theta + \frac{1}{\beta}\psi \right) \end{pmatrix}, \quad (32)$$

so for a.e. $t > 0$,

$$\frac{\|\partial E(\theta(t), \psi(t))\|^2}{4} = \left\| (1 + \alpha\beta)\partial\mathcal{J}(\theta(t)) + (\alpha - \frac{1}{\beta})w(t) \right\|^2 + \left\| \frac{1}{\beta}w(t) \right\|^2. \quad (33)$$

We wish to find $c_1 > 0$, such that $\frac{1}{2}\frac{dE}{dt} + \frac{c_1}{4}\|\partial E\|^2 < 0$. This follows from the following claim.

Claim: let $r_1 > 0$, $r_2 \in \mathbb{R}$, $r_3 > 0$, then there exist C_1 and C_2 two positive constants such that for any $a, b \in \mathbb{R}$,

$$C_1(a^2 + b^2) \leq (r_1a + r_2b)^2 + r_3b^2 \leq C_2(a^2 + b^2). \quad (34)$$

Indeed, the function $Q : (a, b) \mapsto (r_1a + r_2b)^2 + r_3b^2$ is a positive definite quadratic form, C_1 and C_2 can be taken to be two eigenvalues of the positive definite matrix which represents Q . Hence (34) holds for all a and b .

Applying the previous claim to (33) and (31) leads to the existence of $c_1 > 0$ such that for a.e. $t > 0$,

$$\frac{dE}{dt}(\theta(t), \psi(t)) \leq -c_1\|\partial E(\theta(t), \psi(t))\|^2,$$

so assumption (i) holds for INNA.

It now remains to show that (ii) of Theorem 4.2 holds i.e., that there exists $c_2 > 0$ such that for (θ, ψ) solution of (7) and for a.e. $t > 0$, $\|\partial E(\theta(t), \psi(t))\|^2 \geq c_2 \left(\|\dot{\theta}(t)\|^2 + \|\dot{\psi}(t)\|^2 \right)$. Using (7) and (33) we obtain:

$$\frac{\|\partial E(\theta(t), \psi(t))\|^2}{4} = \left\| \frac{1}{\beta}(1 + \alpha\beta)\dot{\theta}(t) + \left[\left(\alpha - \frac{1}{\beta} \right) - \frac{1}{\beta}(1 + \alpha\beta) \right] \dot{\psi}(t) \right\|^2 + \frac{1}{\beta^2} \|\dot{\psi}(t)\|^2, \quad (35)$$

and applying the claim (34) again to (35) one can show that there exist $c_2 > 0$, such that for a.e. $t > 0$,

$$\|\partial E(\theta(t), \psi(t))\|^2 \geq c_2 \left(\|\dot{\theta}(t)\|^2 + \|\dot{\psi}(t)\|^2 \right).$$

So assumption (ii) holds for (7). To conclude, we can apply Theorem 4.2 to (7) and the proof is complete. $\square$

Remark 2. (a) Since the discrete algorithm INNA asymptotically resembles its continuous-time version (see the proof of Theorem 3.1), the results above suggest that similar behaviors and rates could be hoped for INNA itself. Yet, these results remain difficult to obtain in the case of DL, in particular in the mini-batch setting because of the noise $(\xi_k)_{k \in \mathbb{N}}$.

(b) The proof above is significantly simpler when $\alpha\beta > 1$ since Alvarez et al. (2002) proved that in this case, (7) is equivalent to a gradient system, thus assumptions (i) and (ii) of Theorem 4.2 instantly hold.

(c) Theorems 4.2 and 4.3 can be adapted to the case where the Clarke subdifferential is replaced by $D\mathcal{J}$, but we do not state it here for the sake of simplicity.

(d) Theorems 4.2 and 4.3 are actually valid by assuming that $\mathcal{J}$ belongs to a polynomially bounded o-minimal structure. One of the most common instance of such structures is the one given by globally subanalytic sets (as illustrated in a example below). We refer to Bolte et al. (2007a) for a definition and further references.

Let us now comment the results of Theorem 4.3. First, we restrained here to semi-algebraic loss functions $\mathcal{J}$, which are a subset of tame loss functions. Most networks, activation functions and dissimilarity measures mentioned in Section 2.2 fall into this category. Nonetheless, the loss functions of the DL experiments of Section 5.2 are not semi-algebraic. Indeed, the dissimilarity measure l used is the cross-entropy: $l(f(x_n, \theta), y_n) = -\sum_{d=1}^D \mathbf{1}_{[y_n]_d=1} \log([f(x_n, \theta)]_d)$. Such a function cannot be described by polynomials and presents a singularity whenever $[f(x_n, \theta)]_d = 0$. Fortunately, due to the numerical precision but also to the “soft-max” functions often used in classification experiments, the outputs of the network f , for inputs restricted to a compact set, have values in $[\varepsilon, 1]$ for some small $\varepsilon > 0$. Therefore, the singularity at 0 is harmless and the cross-entropy acts as a globally subanalytic function. As a consequence the non-smooth Łojasiewicz inequality holds, and the theorems apply (see also numerical experiments).

The rate of convergence of the trajectory in Theorem 4.3 is non-explicit in the sense that the exponent $b > 0$ is unknown in general. In the light of the proof of Theorem 4.2, this exponent depends on the KL exponent a of the Lyapunov function, which is itself hard to determine in practice. However, the intuition is that small exponents a may yield faster convergence rates (indeed, when $a \in (0, 1/2)$ we actually have a linear rate). As an example, for the function: $t \in \mathbb{R} \mapsto |t|^c$ with $c > 1$, the exponent at 0 is $a = 1 - \frac{1}{c}$ and thus, the closer c is to 1, the smaller a is, and the faster the convergence becomes.

5 Experiments

In this section we first discuss the role and influence of the hyper-parameters of INNA using the 2D example given in Figure 1. We then compare INNA with SGD, ADAGRAD and ADAM on deep learning problems for image recognition.

5.1 Understanding the Role of the Hyper-parameters of INNA

Both hyper-parameters α and β can be seen as damping coefficients from the viewpoint of mechanics as discussed by Alvarez et al. (2002) and sketched in the introduction. Recall the second-order time continuous dynamics which served as a model to the design of INNA:

$$\ddot{\theta}(t) + \alpha \dot{\theta}(t) + \beta \nabla^2 \mathcal{J}(\theta(t)) \dot{\theta}(t) + \nabla \mathcal{J}(\theta(t)) = 0.$$

This differential equation was inspired by Newton’s second law of dynamics asserting that the acceleration of a material point coincides with the sum of forces applied to the particle. As recalled in the introduction three forces are at stake: the gravity and two friction terms. The parameter α calibrates the *viscous damping* intensity as in the Heavy Ball friction method of Polyak (1964). It acts as a dissipation term but it can also be seen as a proximity parameter of the system with the usual gradient descent: the higher α is, the more DIN behaves like a pure gradient descent.⁵ On the other hand the parameter β can be seen as a *Newton damping* which takes into account the geometry of the landscape to brake or accelerate the dynamics in an adaptive anisotropic fashion, see Alvarez and Pérez (1998), Alvarez et al. (2002) for further insights.

We now turn our attention to INNA, and illustrate the versatility of the hyper-parameters α and β in this case. We proceed on a 2D visual non-smooth ill-conditioned example à la Rosenbrock, see Figure 1. For this example, we aim to find the minimum of the function $\mathcal{J}(\theta_1, \theta_2) = 100(\theta_2 - |\theta_1|)^2 + |1 - \theta_1|$. This function has a V-shaped valley, and a unique critical point at (1, 1) which is also the global minimum. Starting from the point (−1, 1.5) (the black cross), we apply INNA with constant steps $\gamma_k = 10^{-4}$. Figure 1 shows that when β is too small, the trajectory presents many transverse oscillations as well as longitudinal ones close to the critical point (subplot a). Then, increasing β significantly reduces transverse oscillations (subplot b). Finally, the longitudinal oscillations are reduced by choosing a higher α (subplot c). In addition, these behaviors are also reflected in the values of the objective function (subplot d). The orange curve (first setting) presents large oscillations. Moreover, looking at the red curve, corresponding to plot (c), there is a short period between 20,000 and 60,000 iterations when the decrease is slower than for the other values of α and β , but still it presents fewer oscillations. In the longer term, the third choice ($\alpha = 1.3$, $\beta = 0.1$) provides remarkably good performance.

The choice of these hyper-parameters may come with rates of convergence for convex and strongly convex smooth functions (Attouch et al., 2020). Following this work, one may also consider to make α and β vary in time (for example like the famous Nesterov damping coefficient $\frac{\alpha}{t}$). In our DL experiments we will however keep these parameters constant so that our theorems still hold. Yet, different behaviors depending on (α, β) can also be observed for DL problems as illustrated on Figure 2 and described next. Although we did not evidence some universal method to choose (α, β) , we used mechanical intuitions to tune these parameters. The coefficient α induces viscous damping, thus one may try to reduce it when convergence appears to be slow. On the other hand, one may want to increase β when large oscillations are observed. Yet, since β affects directly the subgradient effect in (12), taking β too large may jeopardize the numerical

⁵This is easier to see when one rescales $\mathcal{J}$ by α .

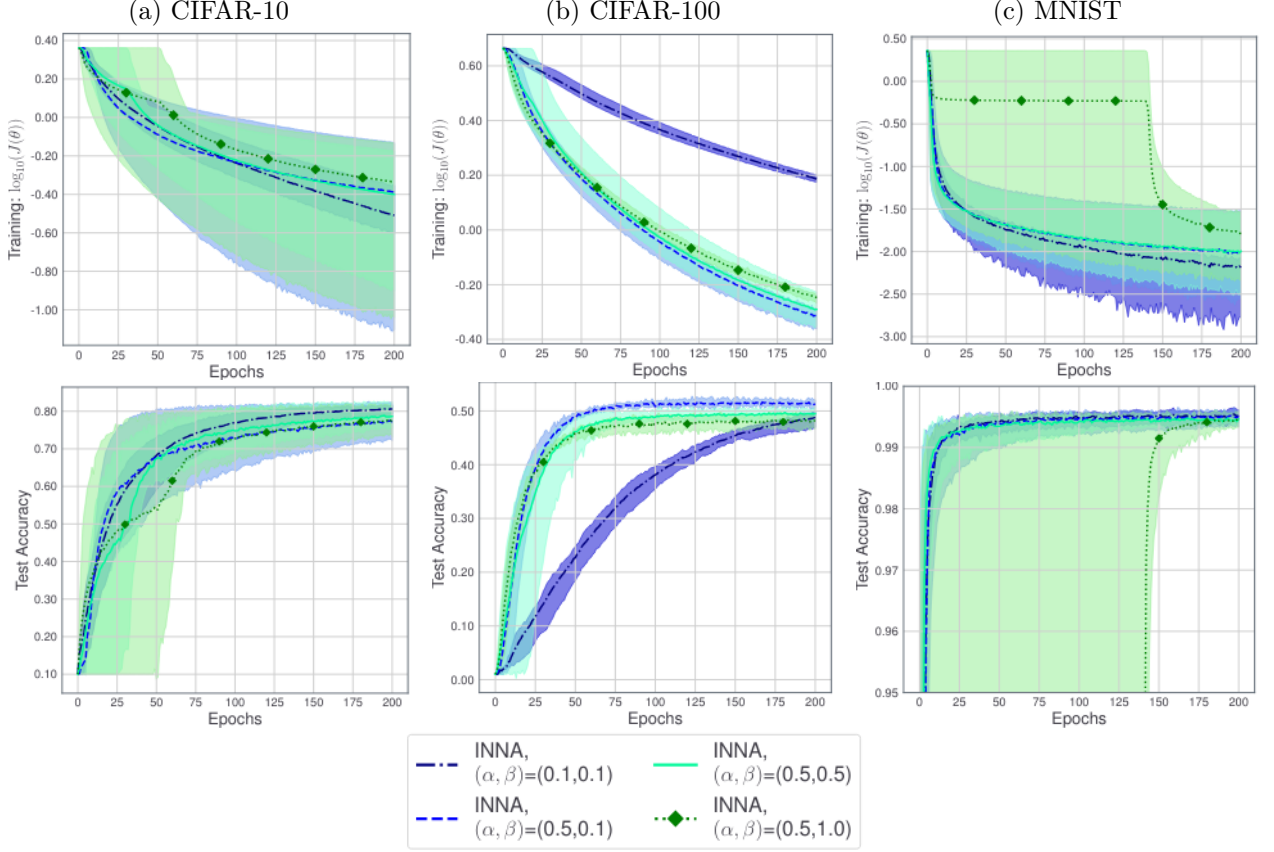


Figure 2: Analysis of the sensibility of INNA to the choice of α and β using NiN for three different image classification problems. Top: logarithm of the loss function $\mathcal{J}(\theta)$ during the training. Bottom: classification accuracy on the test set.

stability of the algorithm. It would be interesting to relate the roles of these coefficients with more structural aspects of the dynamics. Indeed, as mentioned in Remark 2-b, when $\alpha\beta \geq 1$, (11) can be shown to be a gradient system. On the other hand, when $\alpha\beta < 1$ the dynamics is of a different type, we refer to Alvarez et al. (2002) for further comments on this matter.

5.2 Training a DNN with INNA

Before comparing INNA to state-of-the-art algorithms in DL, we first describe the methodology that we followed.

5.2.1 Methodology

- We train a DNN for classification using the three most common image data sets (MNIST, CIFAR-10, CIFAR-100) (LeCun et al., 1998, Krizhevsky, 2009). These data sets are composed of 60,000 small images associated with a label (numbers, objects, animals, etc.). We split the data sets into 50,000 images for training and 10,000 for testing.
- Regarding the network, we use a slightly modified version of the popular Network in Network (NiN) (Lin et al., 2014). It is a reasonably large convolutional network with $P \sim 10^6$ parameters to optimize. We use ReLU activation functions.

- The dissimilarity measure l that is used in the empirical loss $\mathcal{J}$ given by (3) is set to the cross-entropy. The loss function $\mathcal{J}$ is optimized with respect to θ (the weights of the DNN) on the training data. The classification accuracy of the trained DNN is measured using the test data of 10,000 images. Measuring the accuracy boils down to counting how many of the 10,000 were correctly classified (in percentage).
- Based on the results of Section 5.1, we run INNA for four different values of (α, β) :

$$(\alpha, \beta) \in \{(0.1, 0.1), (0.5, 0.1), (0.5, 0.5), (0.5, 1)\}.$$

Given an initialization of the weights θ_0 , we initialize ψ_0 such that the initial velocity is in the direction of $-\nabla \mathcal{J}(\theta_0)$. More precisely, we use $\psi_0 = (1 - \alpha\beta)\theta_0 - (\beta^2 - \beta)\nabla \mathcal{J}(\theta_0)$.

- We compare our algorithm INNA with the classical SGD algorithm and the popular ADAGRAD (Duchi et al., 2011) and ADAM (Kingma and Ba, 2015) algorithms. At each iteration k , we compute the approximation of $\partial \mathcal{J}(\theta)$ on a subset $B_k \subset \{1, \dots, 50,000\}$ of size 32. The algorithms are initialized with the same random weights (drawn from a normal distribution). Five random initializations are considered for each experiment.
- Regarding the selection of step-sizes, ADAGRAD and ADAM both use an adaptive procedure based on past gradients, see Duchi et al. (2011), Kingma and Ba (2015). For the other two algorithms (INNA and SGD), we use the classical step-size schedule $\gamma_k = \frac{\gamma_0}{\sqrt{k+1}}$, which meets Assumption 1. For all four algorithms, choosing the right initial step length γ_0 is often critical in terms of efficiency. We choose this γ_0 using a grid-search: for each algorithm we select the initial step-size that most decreases the training error $\mathcal{J}$ after fifteen epochs (one epoch consisting in a complete pass over the data). The test data is not used to choose the initial step-size nor other hyper-parameters. Note that we could use more flexible step-size schedules but chose a standard schedule for simplicity. Other decay schemes are considered in Figure 4.

For these experiments, we used `Keras 2.2.4` (Chollet, 2015) with `Tensorflow 1.13.1` (Abadi et al., 2016) as backend. The INNA algorithm is available in Pytorch, Keras and Tensorflow: <https://github.com/camcastera/Inna-for-DeepLearning/> (Castera, 2019).

5.2.2 Results

Figure 2 displays the training loss $\mathcal{J}$ and test accuracy with respect to epochs for INNA in its four hyper-parameter configurations considered and for the three data sets considered. Figure 3 displays the performance of INNA with the hyper-parameter configuration that led to the smallest average training error in Figure 2, with comparison to SGD, ADAGRAD and ADAM. In these two figures (and also in subsequent Figure 4), solid lines represent mean values and pale surfaces represent the best and worst runs in terms of training loss and validation accuracy over five random initializations.

Figure 2 suggests that the tuning of the hyper-parameters α and β is not crucial to obtain satisfactory results both for training and testing. It mostly affects the training speed. Thus, INNA looks quite stable with respect to these hyper-parameters. Setting $(\alpha, \beta) = (0.5, 0.1)$ appears to be a good default choice when necessary. Nevertheless, tuning these hyper-parameters is of course advised to get the most from INNA.

Figure 3 shows that best performing methods achieve state-of-the art accuracy using NiN and represent what can be achieved with a moderately large network and coarse grid-search

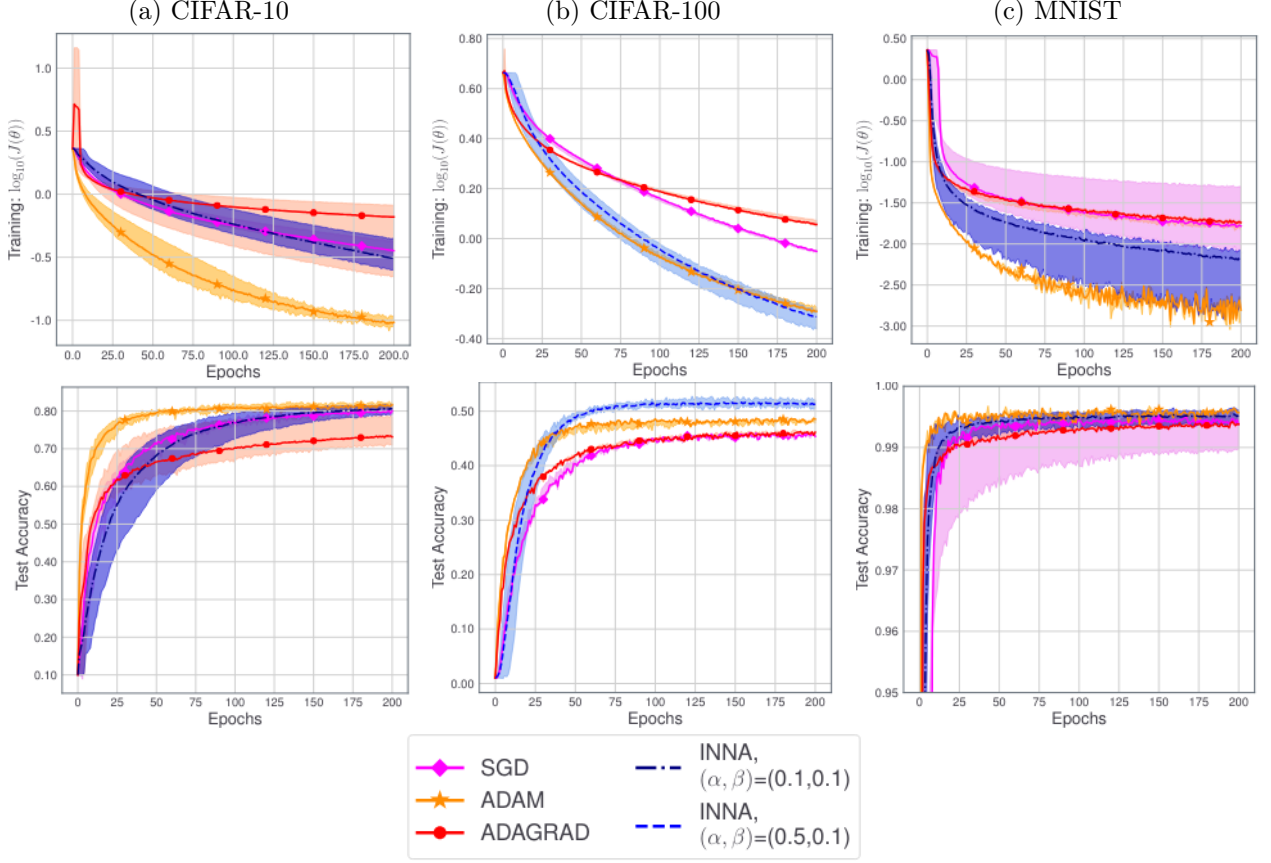


Figure 3: Comparison of INNA with state-of-the-art algorithms SGD, ADAM and ADAGRAD. Top: logarithm of the loss function $\mathcal{J}(\theta)$ during the training. Bottom: classification accuracy on the test set.

tuning of the initial step-size. In our comparison, INNA and ADAM outperform SGD and ADAGRAD for training. While ADAM seems to be faster in the early training phase, INNA achieves the best accuracy almost every time especially on CIFAR-100 (Figure 3(b)). Thus, INNA appears to be competitive in comparison to other algorithms with the advantage of having solid theoretical foundations and a simple step-size rule as compared to ADAM and ADAGRAD.

Finally, let us point out that although ADAM was faster in the experiments of Figure 3, INNA can outperform ADAM using the slow step-size decay discussed in Section 3.2. Indeed, in the previous experiments we used a standard decreasing step-size of the form $\gamma_0/\sqrt{k+1}$ for simplicity, but Assumption 1 allows for step-sizes decreasing much more slowly. As such, we also considered decays of the form $\gamma_0(k+1)^{-q}$ with $q \leq 1/2$. The results are displayed on top of Figure 4. Except when q is too small (too slow decay, e.g., $q = 1/16$), these results show that some decays slower than $q = 1/2$ make INNA a little faster than any of the other algorithms we tried. In particular, with a step-size decay proportional to $k^{-1/4}$, INNA outperforms ADAM (bottom of Figure 4). This suggests that tuning q can also significantly accelerate the training process.

6 Conclusion

We introduced a novel stochastic optimization algorithm featuring inertial and Newtonian behavior motivated by applications to deep learning. We provided a powerful algorithmic convergence

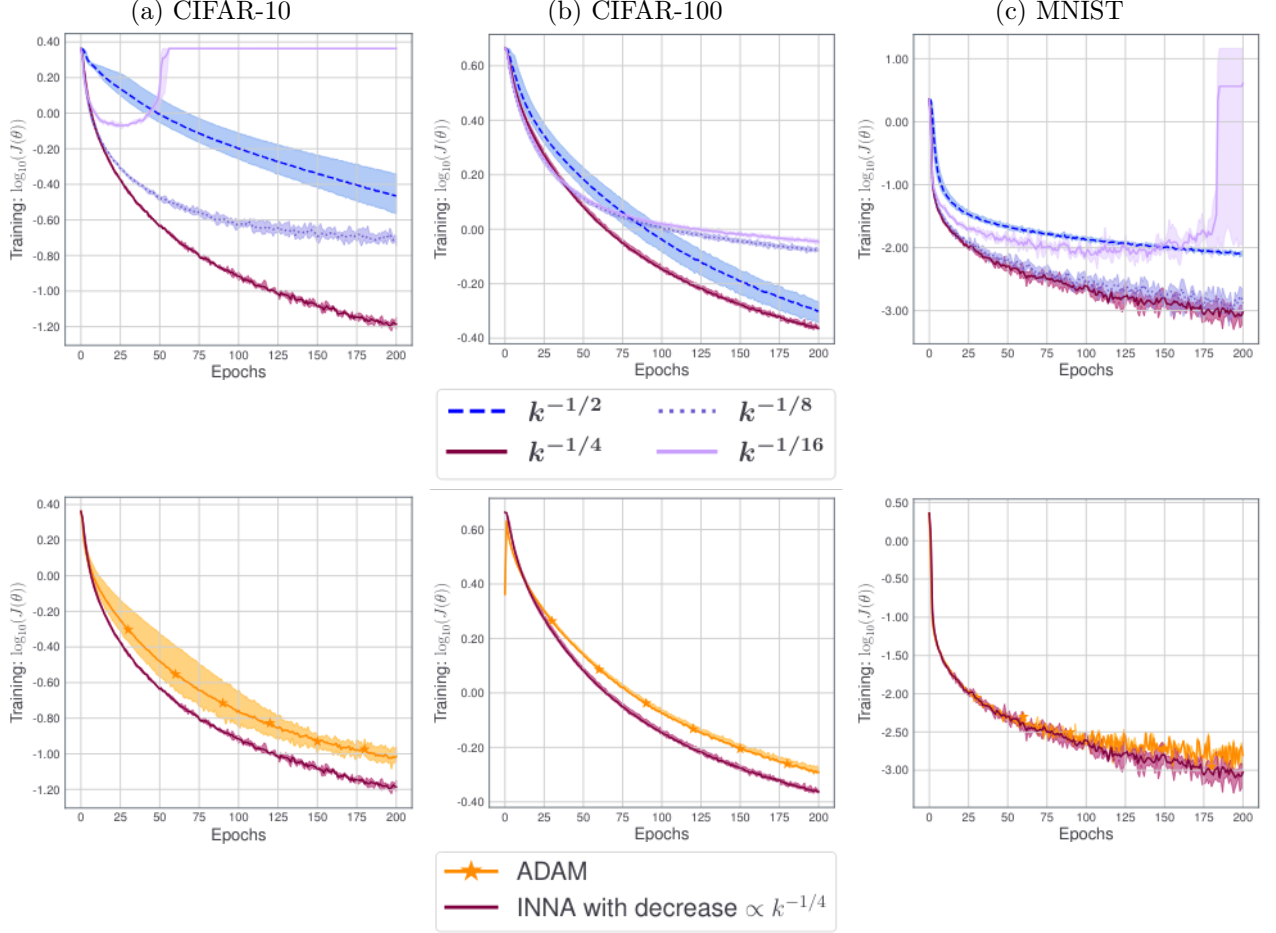


Figure 4: On top: Training loss of INNA on three image classification problems with various step-size decays. In the legend, k^{-q} means a step-size decay at iteration k of the form $\gamma_k = \gamma_0 k^{-q}$. The bottom row show the comparison between INNA with a well-chosen step-size decay and ADAM.

analysis under weak hypotheses applicable to most DL problems. We also provided new general results to study differential inclusions on Clarke subdifferential and obtain convergence rates for the continuous-time counterpart of our algorithm. We would like to point out that, apart from SGD (Davis et al., 2020), the convergence of concurrent methods in such a general setting is still an open question. Our result seems moreover to be the first one to be able to rigorously handle the analysis of mini-batch sub-sampling for ReLU DNNs via the introduction of the D -critical points. Our experiments show that INNA is very competitive with state-of-the-art algorithms for DL but also very malleable. We stress that these numerical manipulations were performed on substantial DL benchmarks with only limited algorithm tuning. This facilitates reproducibility and allows staying as close as possible to the reality of DL applications in machine learning.

Acknowledgments

The authors acknowledge the support of the European Research Council (ERC FACTORY-CoG-6681839), the Agence Nationale de la Recherche (ANR 3IA-ANITI, ANR-17-EURE-0010 CHESS, ANR-19-CE23-0017 MASDOL) and the Air Force Office of Scientific Research (FA9550-18-1-0226).

Part of the numerical experiments were done using the OSIRIM platform of IRIT, supported by the CNRS, the FEDER, Région Occitanie and the French government (<http://osirim.irit.fr/site/en>). We thank the development teams of the following libraries that were used in the experiments: Python (Rossum, 1995), Numpy (Walt et al., 2011), Matplotlib (Hunter, 2007), Pytorch (Paszke et al., 2019), Tensorflow and Keras (Abadi et al., 2016, Chollet, 2015).

The authors thank the anonymous reviewers for their comments which helped to improve the paper and thank Hedy Attouch and Sixin Zhang for useful discussions.

References

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., et al. (2016). Tensorflow: A system for large-scale machine learning. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283.
- Adil, S. (2018). *Opérateurs monotones aléatoires et application à l’optimisation stochastique*. PhD Thesis, Paris Saclay.
- Alvarez, F., Attouch, H., Bolte, J., and Redont, P. (2002). A second-order gradient-like dissipative dynamical system with Hessian-driven damping: Application to optimization and mechanics. *Journal de Mathématiques Pures et Appliquées*, 81(8):747–779.
- Alvarez, F. and Pérez, J. M. (1998). A dynamical system associated with Newton’s method for parametric approximations of convex minimization problems. *Applied Mathematics and Optimization*, 38:193–217.
- Attouch, H., Bolte, J., Redont, P., and Soubeyran, A. (2010). Proximal alternating minimization and projection methods for nonconvex problems: An approach based on the Kurdyka-Łojasiewicz inequality. *Mathematics of Operations Research*, 35(2):438–457.
- Attouch, H., Chbani, Z., Fadili, J., and Riahi, H. (2020). First-order optimization algorithms via inertial systems with Hessian driven damping. *Mathematical Programming*.
- Aubin, J.-P. and Cellina, A. (2012). *Differential inclusions: set-valued maps and viability theory*. Springer.
- Barakat, A. and Bianchi, P. (2021). Convergence and dynamical behavior of the ADAM algorithm for nonconvex stochastic optimization. *SIAM Journal on Optimization*, 31(1):244–274.
- Benaïm, M. (1999). Dynamics of stochastic approximation algorithms. In *Séminaire de Probabilités XXXIII*, pages 1–68. Springer.
- Benaïm, M., Hofbauer, J., and Sorin, S. (2005). Stochastic approximations and differential inclusions. *SIAM Journal on Control and Optimization*, 44(1):328–348.
- Berahas, A. S., Bollapragada, R., and Nocedal, J. (2020). An investigation of Newton-sketch and subsampled Newton methods. *Optimization Methods and Software*, 35(4):661–680.
- Bianchi, P., Hachem, W., and Schechtman, S. (2020). Convergence of constant step stochastic gradient descent for non-smooth non-convex functions. *arXiv preprint arXiv:2005.08513*.

- Bolte, J., Daniilidis, A., and Lewis, A. (2007a). The Łojasiewicz inequality for nonsmooth subanalytic functions with applications to subgradient dynamical systems. *SIAM Journal on Optimization*, 17(4):1205–1223.
- Bolte, J., Daniilidis, A., Lewis, A., and Shiotani, M. (2007b). Clarke subgradients of stratifiable functions. *SIAM Journal on Optimization*, 18(2):556–572.
- Bolte, J., Daniilidis, A., Ley, O., and Mazet, L. (2010). Characterizations of Łojasiewicz inequalities: subgradient flows, talweg, convexity. *Transactions of the American Mathematical Society*, 362(6):3319–3363.
- Bolte, J. and Pauwels, E. (2020a). Conservative set valued fields, automatic differentiation, stochastic gradient methods and deep learning. *Mathematical Programming*, pages 1–33.
- Bolte, J. and Pauwels, E. (2020b). A mathematical model for automatic differentiation in machine learning. In *Advances in Neural Information Processing Systems (NIPS)*.
- Bolte, J., Sabach, S., and Teboulle, M. (2014). Proximal alternating linearized minimization for nonconvex and nonsmooth problems. *Mathematical Programming*, 146(1-2):459–494.
- Borkar, V. S. (2009). *Stochastic approximation: A dynamical systems viewpoint*. Springer.
- Bottou, L. and Bousquet, O. (2008). The tradeoffs of large scale learning. In *Advances in Neural Information Processing Systems (NIPS)*, pages 161–168.
- Bottou, L., Curtis, F. E., and Nocedal, J. (2018). Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311.
- Byrd, R. H., Chin, G. M., Neveitt, W., and Nocedal, J. (2011). On the use of stochastic Hessian information in optimization methods for machine learning. *SIAM Journal on Optimization*, 21(3):977–995.
- Byrd, R. H., Hansen, S. L., Nocedal, J., and Singer, Y. (2016). A stochastic quasi-Newton method for large-scale optimization. *SIAM Journal on Optimization*, 26(2):1008–1031.
- Castera, C. (2019). INNA for deep learning. <https://github.com/camcastera/Inna-for-DeepLearning/>.
- Castera, C., Bolte, J., Févotte, C., and Pauwels, E. (2019). An inertial Newton algorithm for deep learning. *arXiv preprint:1905.12278v1*.
- Chollet, F. (2015). Keras. <https://github.com/fchollet/keras>.
- Clarke, F. H. (1990). *Optimization and nonsmooth analysis*. SIAM.
- Coste, M. (2000). *An introduction to o-minimal geometry*. Istituti editoriali e poligrafici internazionali Pisa.
- Davis, D., Drusvyatskiy, D., Kakade, S., and Lee, J. D. (2020). Stochastic subgradient method converges on tame functions. *Foundations of Computational mathematics*, 20(1):119–154.
- van den Dries, L. (1998). *Tame topology and o-minimal structures*. Cambridge university press.
- Duchi, J., Hazan, E., and Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(7):2121–2159.

- Duchi, J. C. and Ruan, F. (2018). Stochastic methods for composite and weakly convex optimization problems. *SIAM Journal on Optimization*, 28(4):3229–3259.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in science & engineering*, 9(3):90–95.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Technical report, Canadian Institute for Advanced Research.
- Kurdyka, K. (1998). On gradients of functions definable in o-minimal structures. In *Annales de l’institut Fourier*, volume 48, pages 769–783.
- Kushner, H. and Yin, G. G. (2003). *Stochastic approximation and recursive algorithms and applications*. Springer.
- LeCun, Y., Bottou, L., Bengio, Y., Haffner, P., et al. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.
- Lin, M., Chen, Q., and Yan, S. (2014). Network in Network. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Ljung, L. (1977). Analysis of recursive stochastic algorithms. *IEEE Transactions on Automatic Control*, 22(4):551–575.
- Martens, J. (2010). Deep learning via Hessian-free optimization. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 735–742.
- Moulines, E. and Bach, F. R. (2011). Non-asymptotic analysis of stochastic approximation algorithms for machine learning. In *Advances in Neural Information Processing Systems (NIPS)*, pages 451–459.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NIPS)*, pages 8026–8037.
- Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17.
- Robbins, H. and Monro, S. (1951). A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(1):400–407.
- Rossum, G. (1995). *Python reference manual*. CWI (Centre for Mathematics and Computer Science).
- Rumelhart, D. E. and Hinton, G. E. (1986). Learning representations by back-propagating errors. *Nature*, 323(9):533–536.
- Shiota, M. (2012). *Geometry of subanalytic and semialgebraic sets*, volume 150. Springer Science & Business Media.

- Walt, S. v. d., Colbert, S. C., and Varoquaux, G. (2011). The NumPy array: a structure for efficient numerical computation. *Computing in science & engineering*, 13(2):22–30.
- Wilson, A. C., Roelofs, R., Stern, M., Srebro, N., and Recht, B. (2017). The marginal value of adaptive gradient methods in machine learning. In *Advances in Neural Information Processing Systems (NIPS)*, pages 4148–4158.
- Xu, P., Roosta, F., and Mahoney, M. W. (2020). Second-order optimization for non-convex machine learning: an empirical study. In *Proceedings of the SIAM International Conference on Data Mining (SDM20)*, pages 199–207. Society for Industrial and Applied Mathematics.